

ECE 667

Spring 2013

Synthesis and Verification of Digital Circuits

Introduction

Design Flow

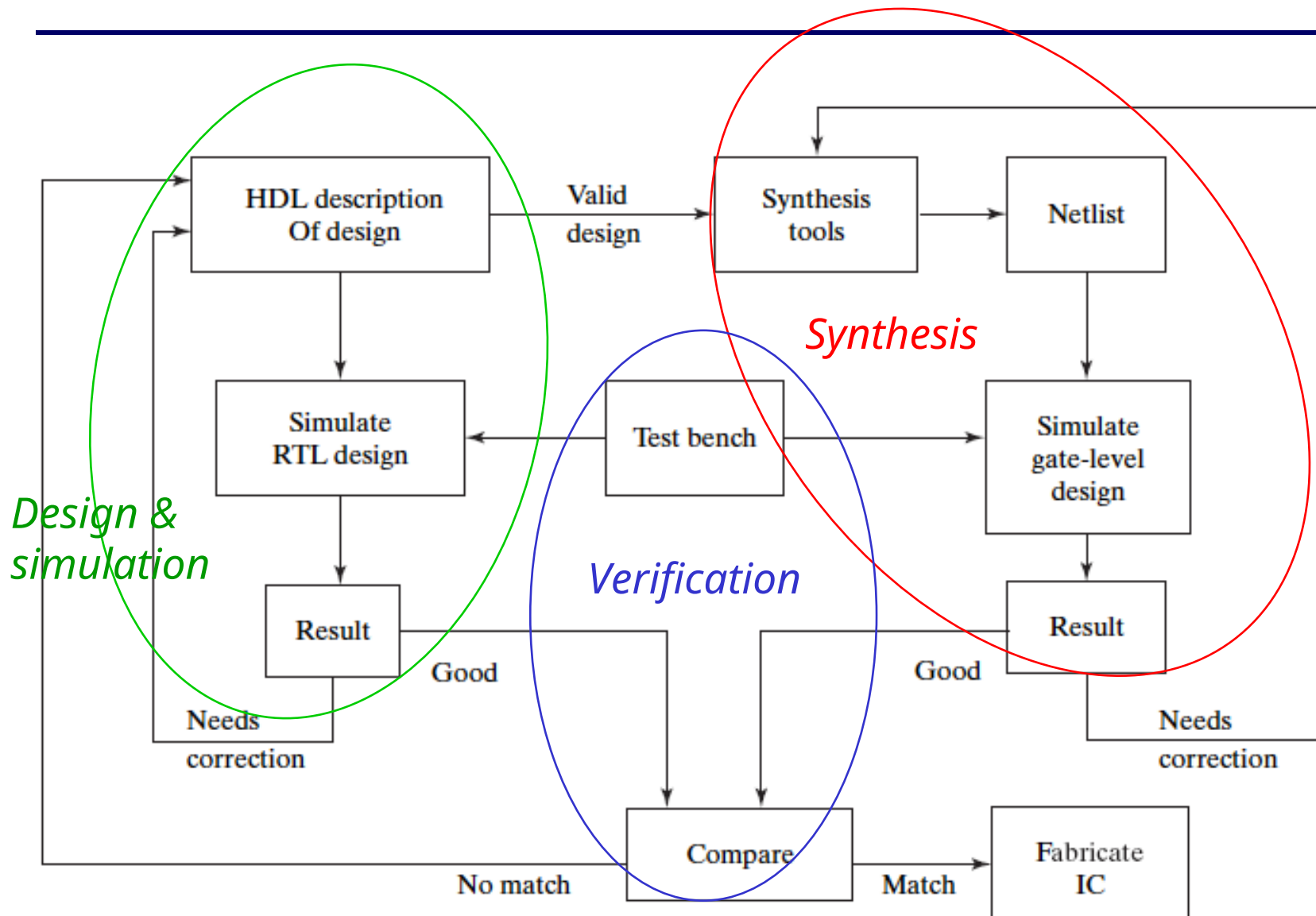
Course Outline

- **Introduction to logic synthesis**
 - VLSI design flow, target technologies
- **High level synthesis, basics**
 - Scheduling, resource allocation, binding
- **Boolean functions and their representations**
 - Sum of products, factored form representations
 - Canonical representations, BDDs, BMDs, others
- **Two-level logic optimization**
 - Exact logic minimization (Quine)
 - Heuristic logic optimization (Espresso)
- **Functional decomposition**
 - Asenhurst-Curtis method
 - BDD based decomposition, bi-decomposition
- **Multi-level logic synthesis (technology independent)**
 - Kernel-based algebraic decomposition (SIS)
 - AIG-based optimization (ABC)
- **Technology mapping**
 - Graph based, standard cell mapping (ASICs)
 - Cut-based (FPGAs)
- **Sequential optimization**
 - Retiming; integrating synthesis, retiming and mapping (ABC)
- **Satisfiability (SAT)**
 - Application to synthesis and verification
- **Formal verification**
 - Equivalence checking, property checking
 - Sequential verification, FSM reachability

Outline – today's lecture

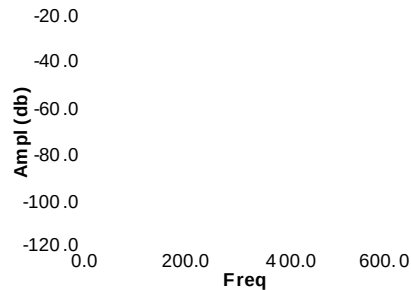
- **Intro: synthesis flow**
 - DataPath (high level synthesis)
 - Control and steering logic (logic synthesis)
- **Target technology**
 - PLA, ASIC, FPGA
- **Logic optimization, objectives**
 - Two-level (PLA)
 - Multi-level (standard cells, FPGAs)
 - Technology independent + mapping
 - Combinational vs sequential logic synthesis
- **Representations**
 - Truth tables, K-maps
 - SoP, factored forms, BDDs

Electronic Design Automation (EDA)

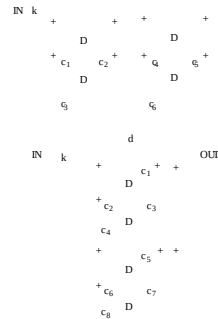


Synthesis Process (high-level view)

APPLICATION



ALGORITHM



ARCHITECTURE

interconnect

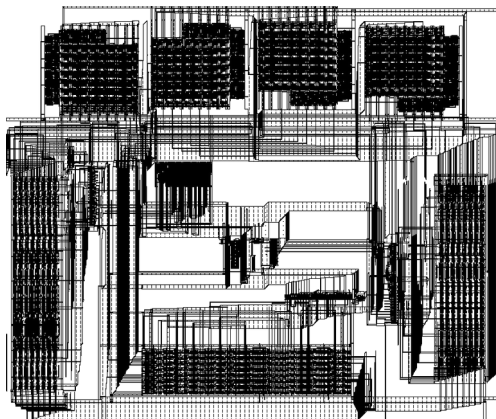
ASIC

GP signal processor

MCM

memory

LOGIC AND PHYSICAL SYNTHESIS



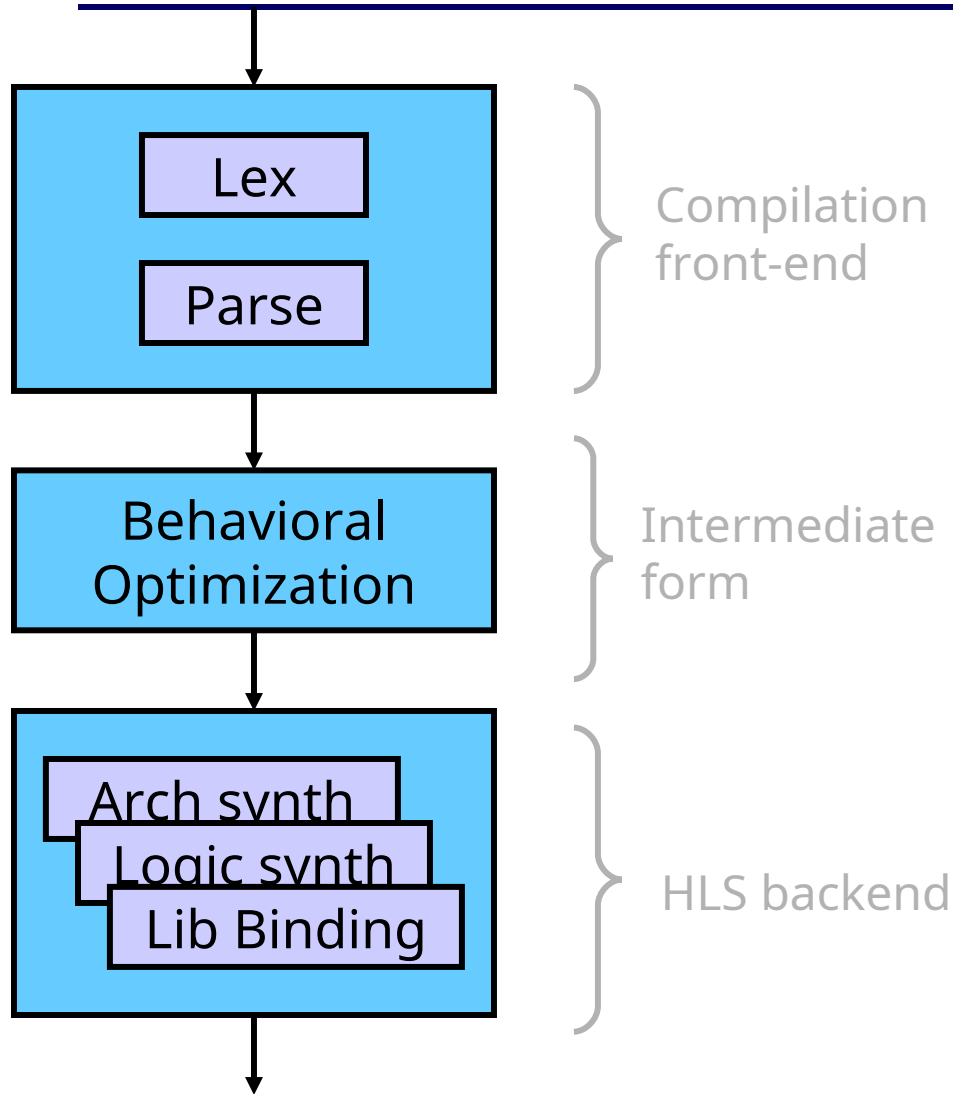
HIGH-LEVEL SYNTHESIS

S1 S2 S3 S4

High Level Synthesis (HLS)

- The process of converting high-level design description to RTL
 - Input:
 - High-level languages (C, system C, system Verilog)
 - Hardware description languages (Verilog, VHDL)
 - State diagrams / logic networks
 - Tools:
 - Parser, compiler
 - Library of modules
 - Constraints:
 - Resource constraints (no. of modules of a certain type)
 - Timing constraints (Latency, delay, clock cycle)
 - Output:
 - Operation scheduling (time) and binding (resource)
 - Control generation
 - RTL architecture

Design Compilation



Separation into

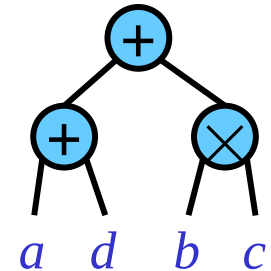
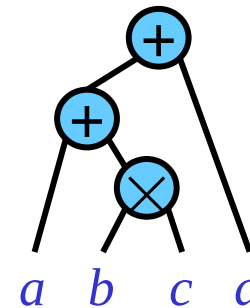
- DataPath (arithmetic)
- Control (Boolean logic)

Behavioral Optimization

- Techniques used in software compilation

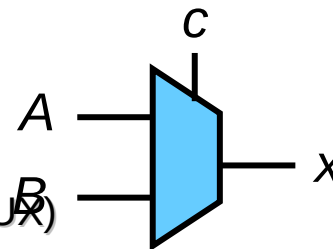
- Expression tree height reduction
- Constant and variable propagation
- Common sub-expression elimination
- Dead-code elimination
- Operator strength reduction (e.g., $*4 \rightarrow \ll 2$)

$$x = a + b \times c + d$$



- Hardware transformations

- Conditional expansion
 - If c then $x = A$
else $x = B$;
 - Compute A and B in parallel: $x = C ? A : B$ (MUX)
- Loop unrolling
 - Replace k iterations of a loop by k instances of the loop body



- Data Flow Graph (DFG) transformations

Example – Digital Filter design

- A second-order digital filter

Verilog code:

/ A behavioral description of a digital filter*

Algorithm:

$$y_1(kh + h) = c.(r_1 + r_2)$$

$$r_1 = x_1(kh) + t_2(kh)$$

$$r_2 = r_1.a_{11} + t_2(kh)$$

$$t_1(kh + h) = r_3$$

$$r_3 = r_4.a_{21} + t_1(kh)$$

$$r_4 = r_2 + t_1(kh)$$

$$t_2(kh + h) = r_3 + r_4$$

```
module digital_filter(x1,y1);
```

```
input x1;
```

```
output y1;
```

```
wire [7:0] r1,r2,r3,r4,t1,t2,c,a11,a21;
```

```
assign r1 = x1 + t2;
```

```
assign r2 = r1 * a11 + t2;
```

```
assign r4 = r2 + t1;
```

```
assign r3 = r4 + a21 + t1;
```

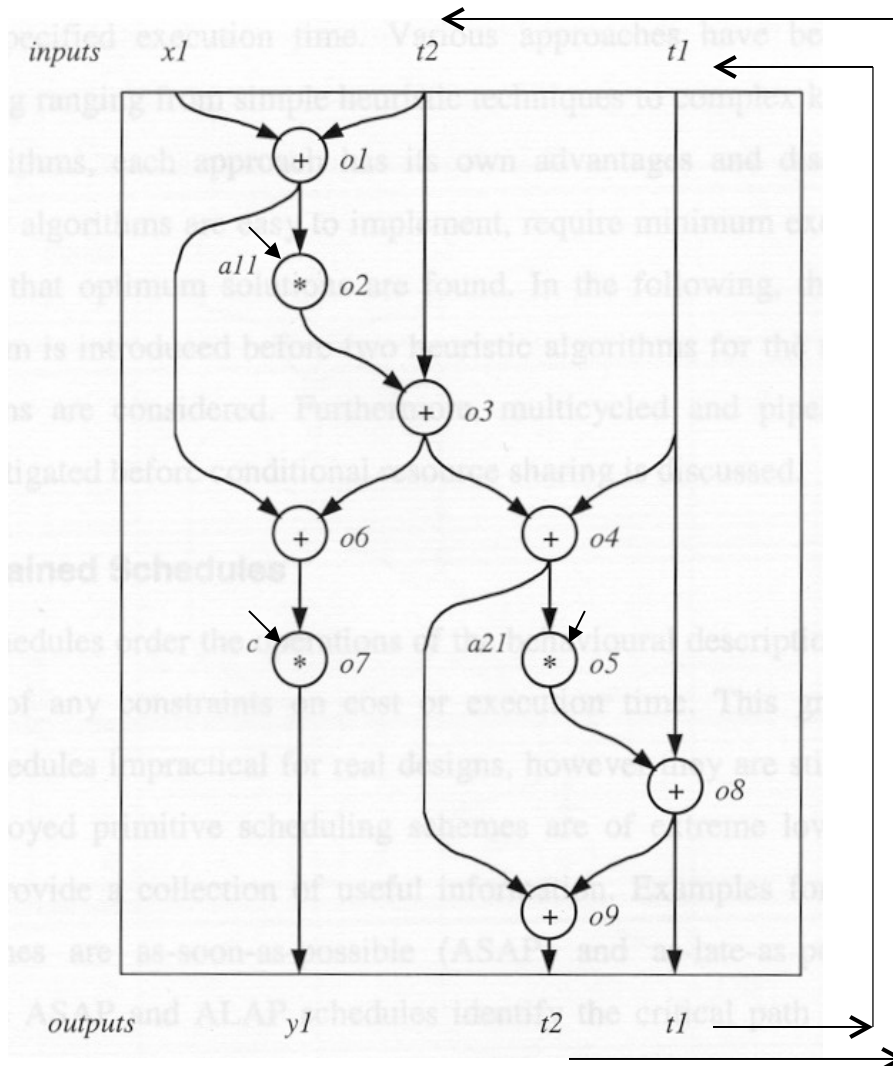
```
assign y1 = c* (r1 + r2);
```

```
assign t1 = r3;
```

```
assign t2 = r3 + r4;
```

```
endmodule
```

Digital Filter – Unscheduled DFG



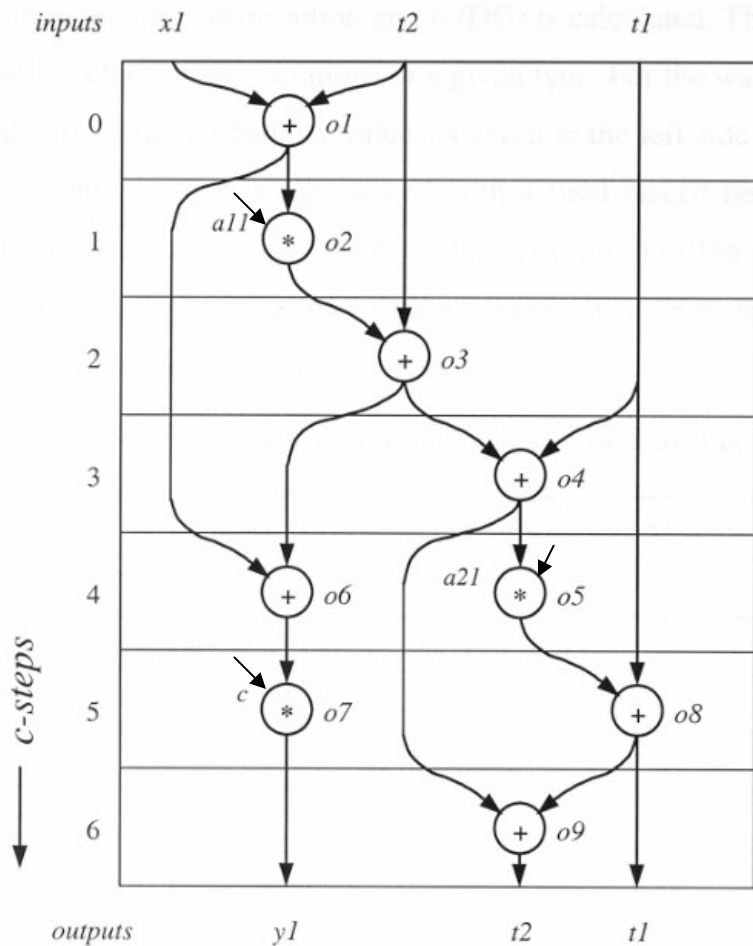
```
input x1;
output y1;
wire [7:0] r1,r2,r3,r4,t1,t2,c,a11,a21;
```

```
assign r1 = x1 + t2;
assign r2 = r1 * a11 + t2;
assign r4 = r2 + t1;
assign r3 = r4 + a21 + t1;
```

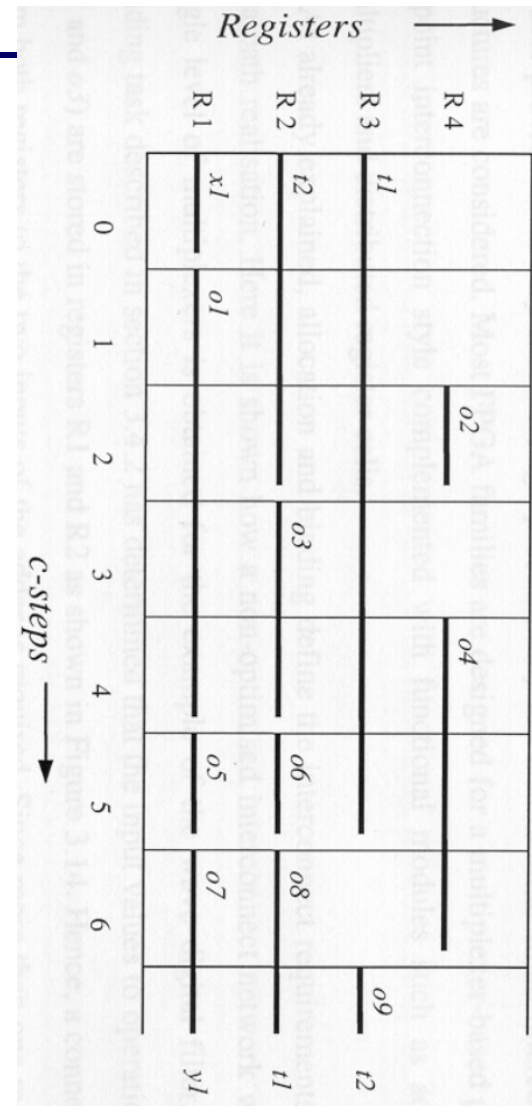
```
assign y1 = c * (r1 + r2);
assign t1 = r3;
assign t2 = r3 + r4;
```

```
endmodule
```

Digital Filter – Scheduling and Regs mapping

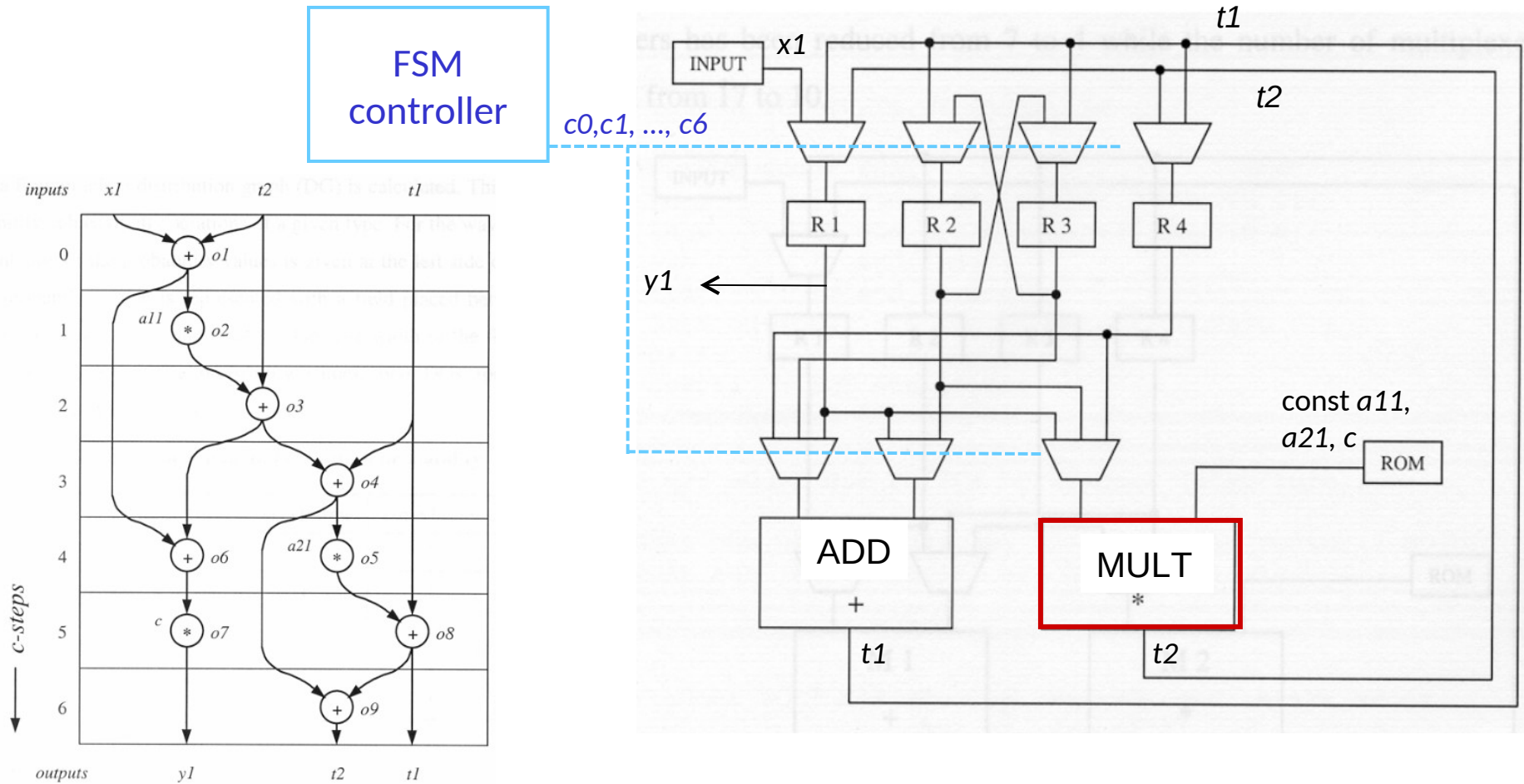


Resource-constraint scheduling (1 adder , 1 multiplier)



Register mapping (left-edge algorithm)

Example – Final Architecture

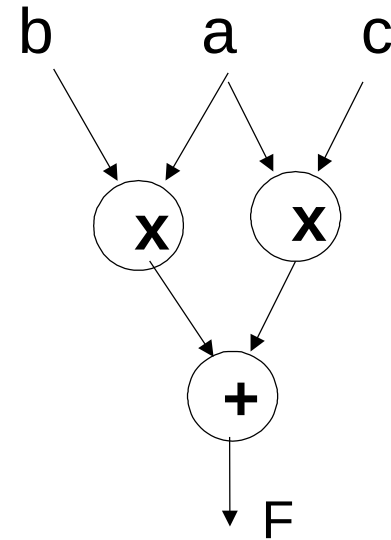
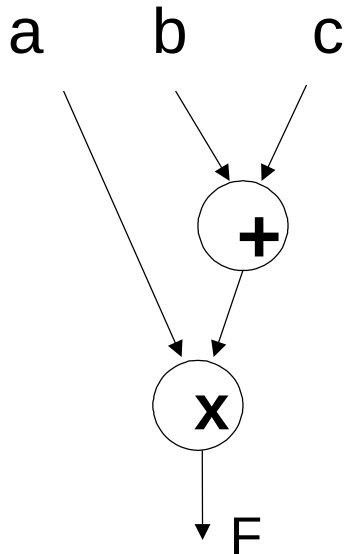


Arithmetic components (structured) → design ware (DC)
 Control + steering logic (unstructured) → logic synthesis

Data Flow Graph (DFG) Transformations

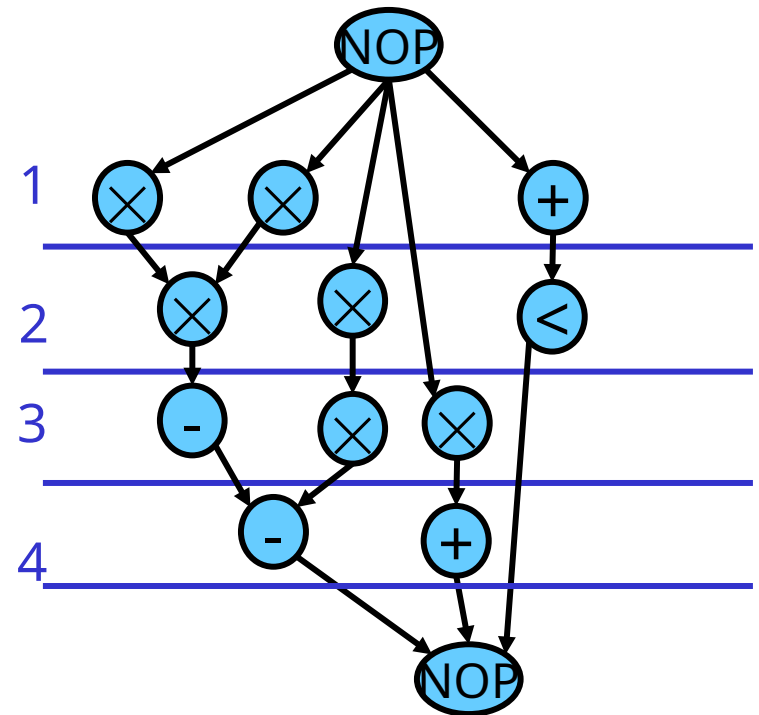
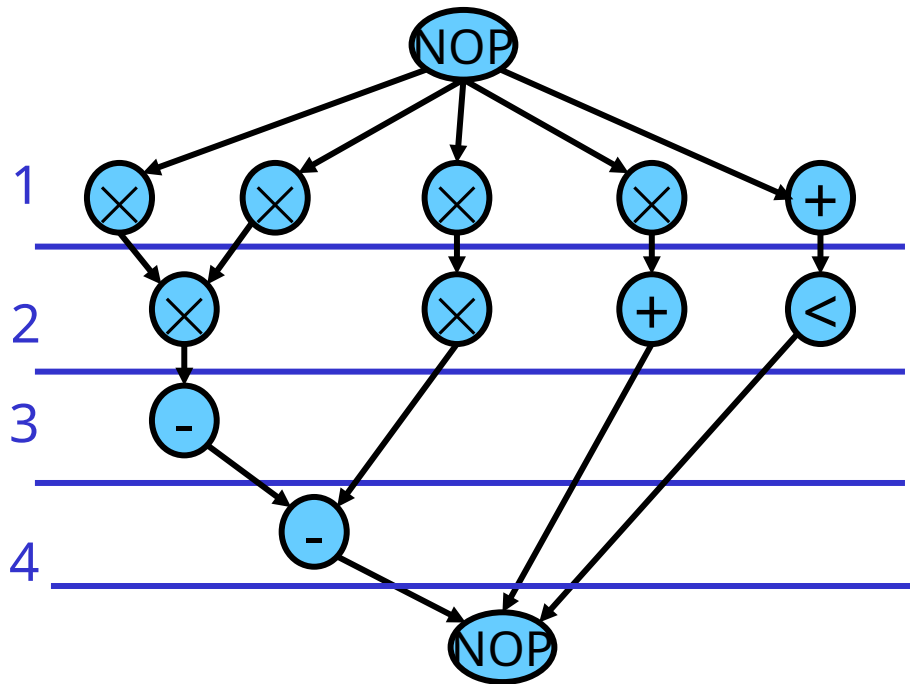
Transformation

$$F = a*(b + c) \longleftrightarrow F = a*b + a*c$$



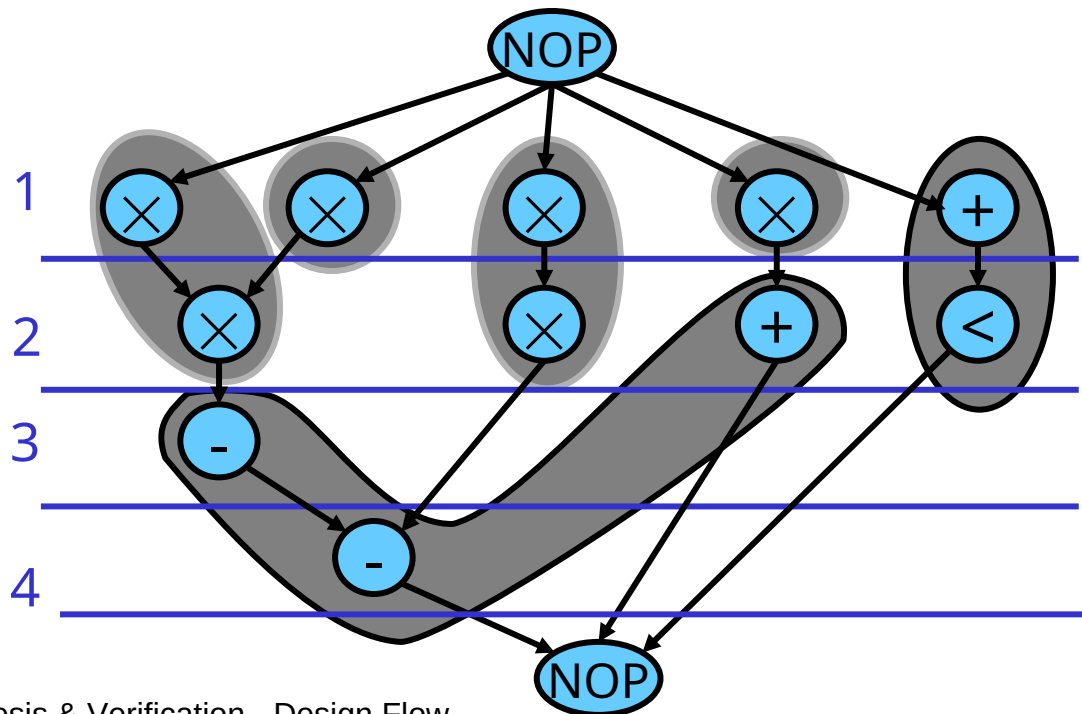
Optimization in Temporal Domain

- **Scheduling:**
 - Mapping of operations to time slots (cycles)
 - Uses sequencing graph (data flow graph, DFG)
 - Goal: minimize latency (s.t. resource constraints)



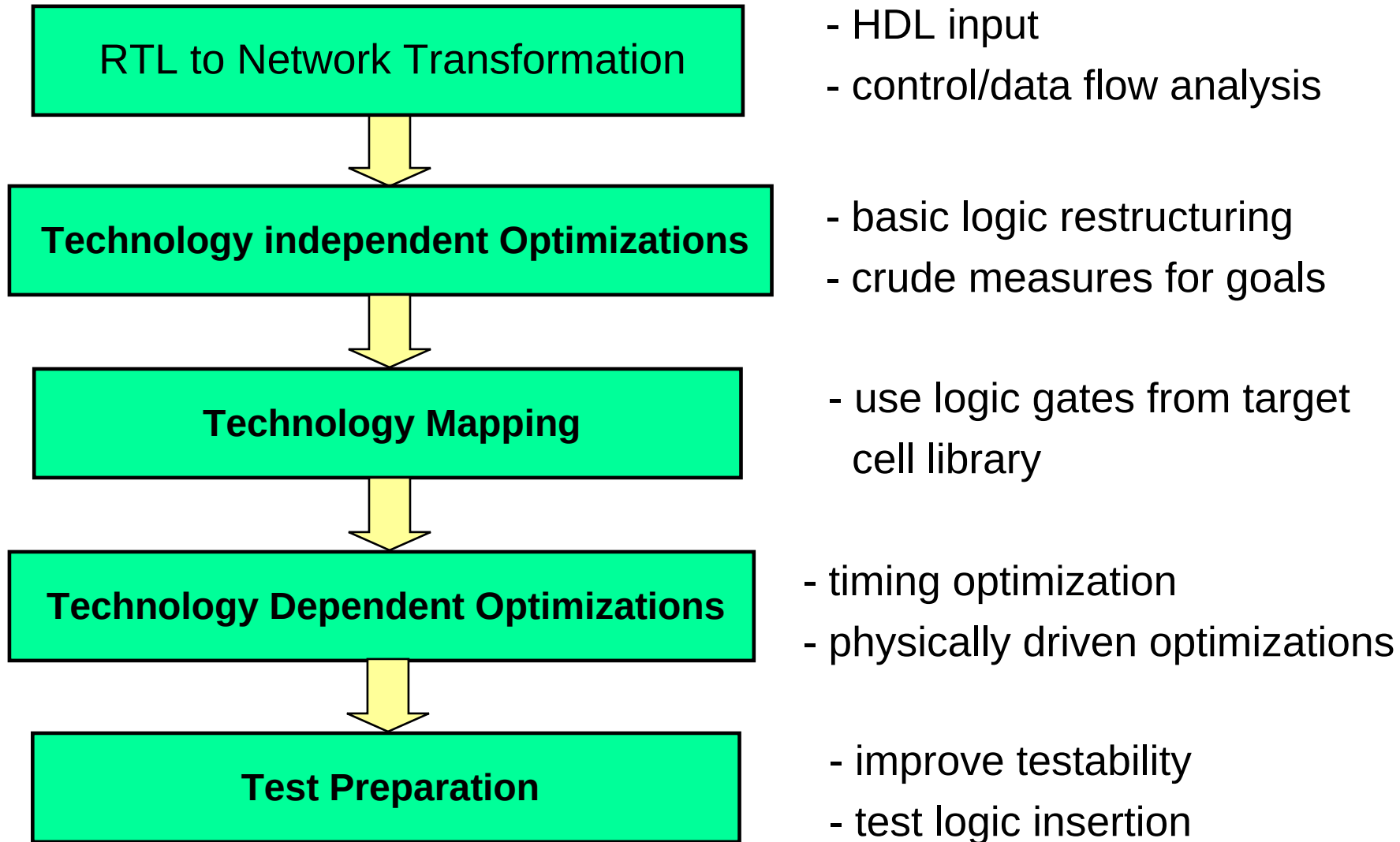
Optimization in Spatial Domain

- **Resource allocation & binding**
 - Assigning operations to hardware units
 - Allocating registers
 - Binding operations to same resource
 - Goal: minimize resource utilization (s.t. latency constraints)



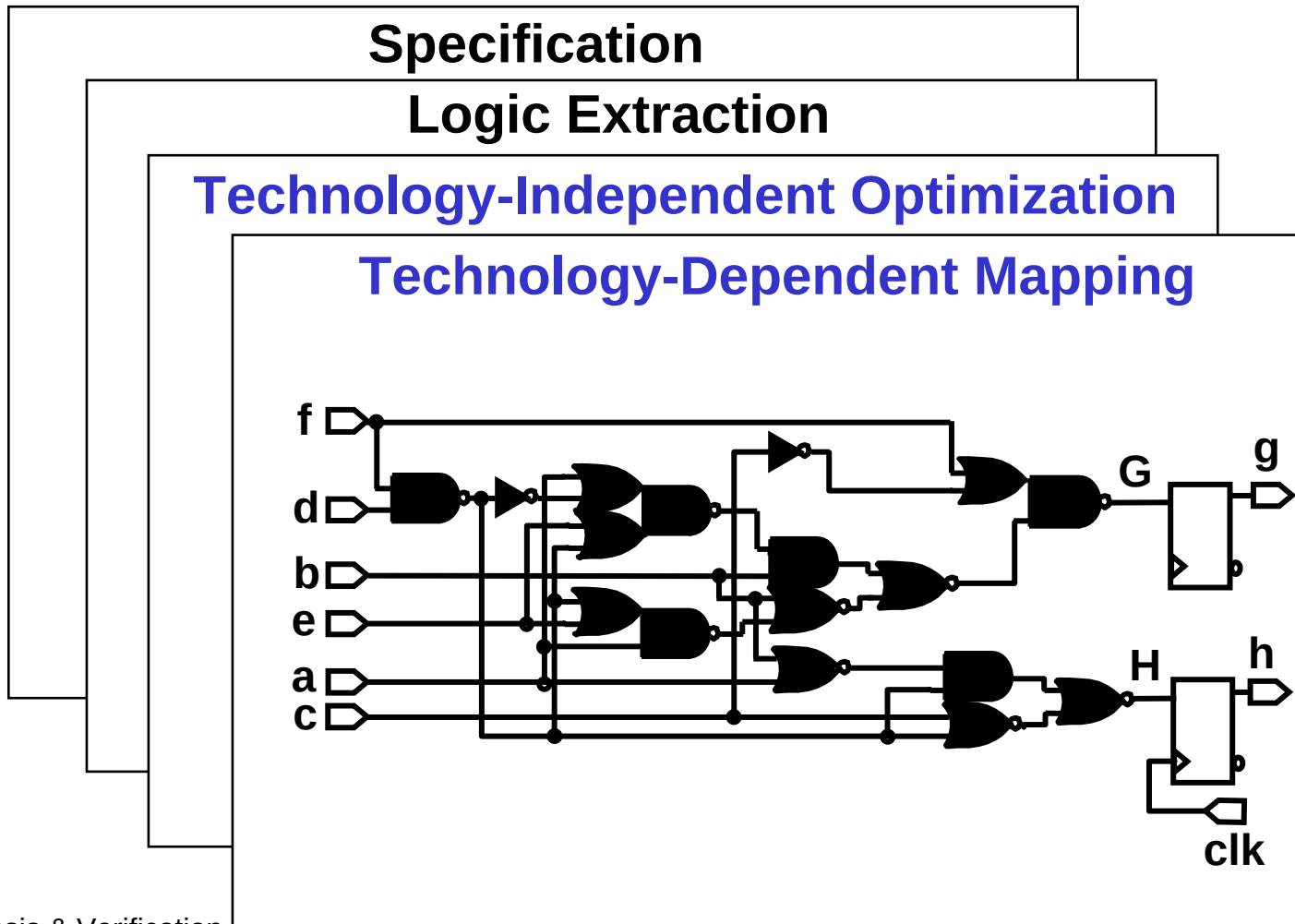
[©Gupta]

Logic (RTL) Synthesis



Synthesis Flow (logic level)

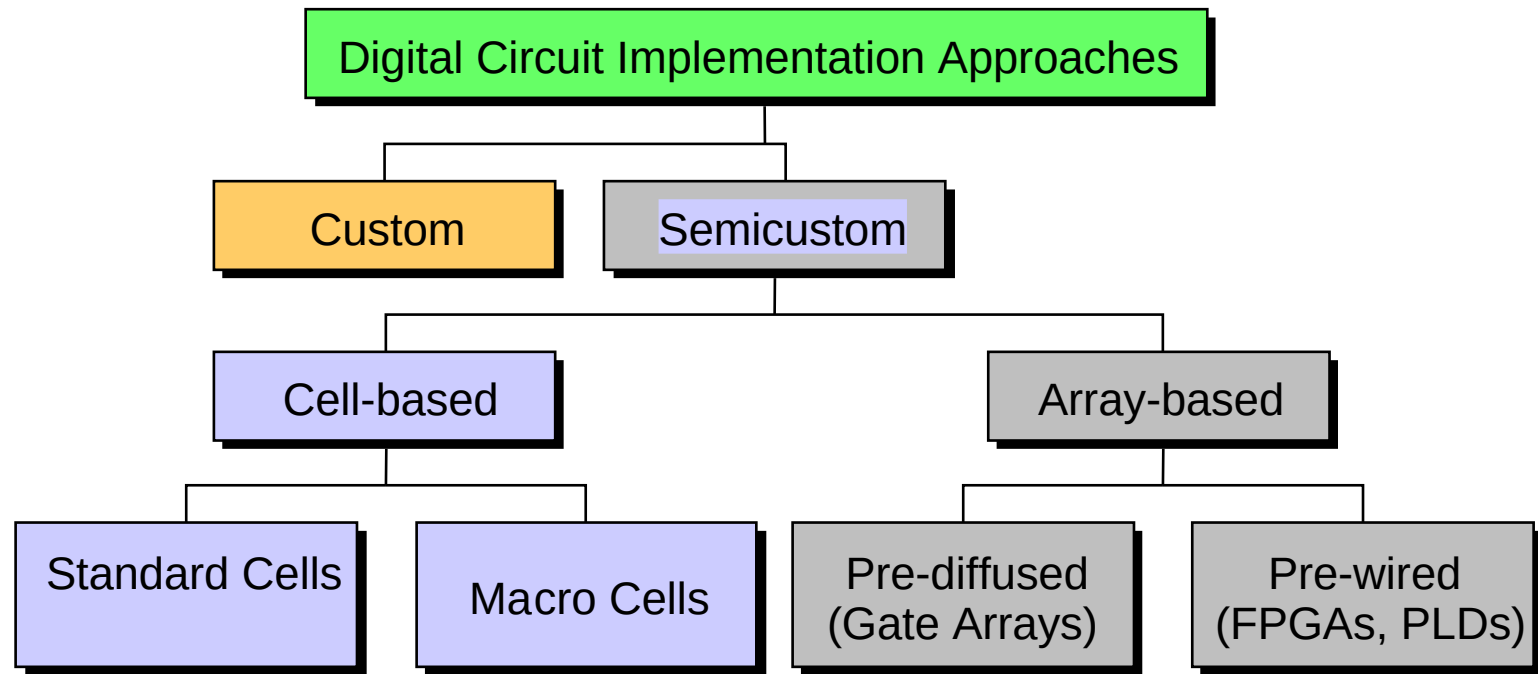
a multi-stage process



Multi-level logic circuit

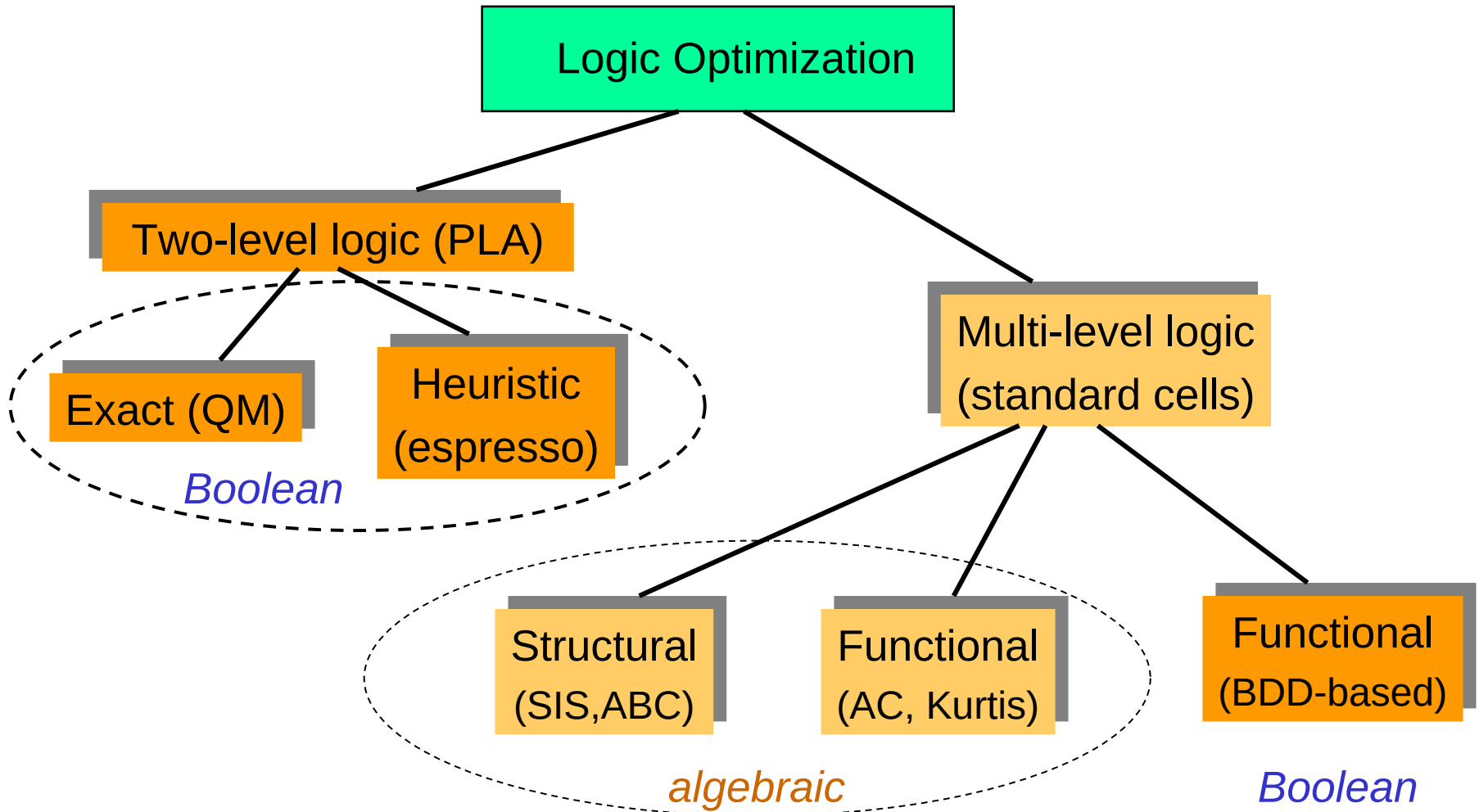
Physical layout

Implementation Choices (target technology)



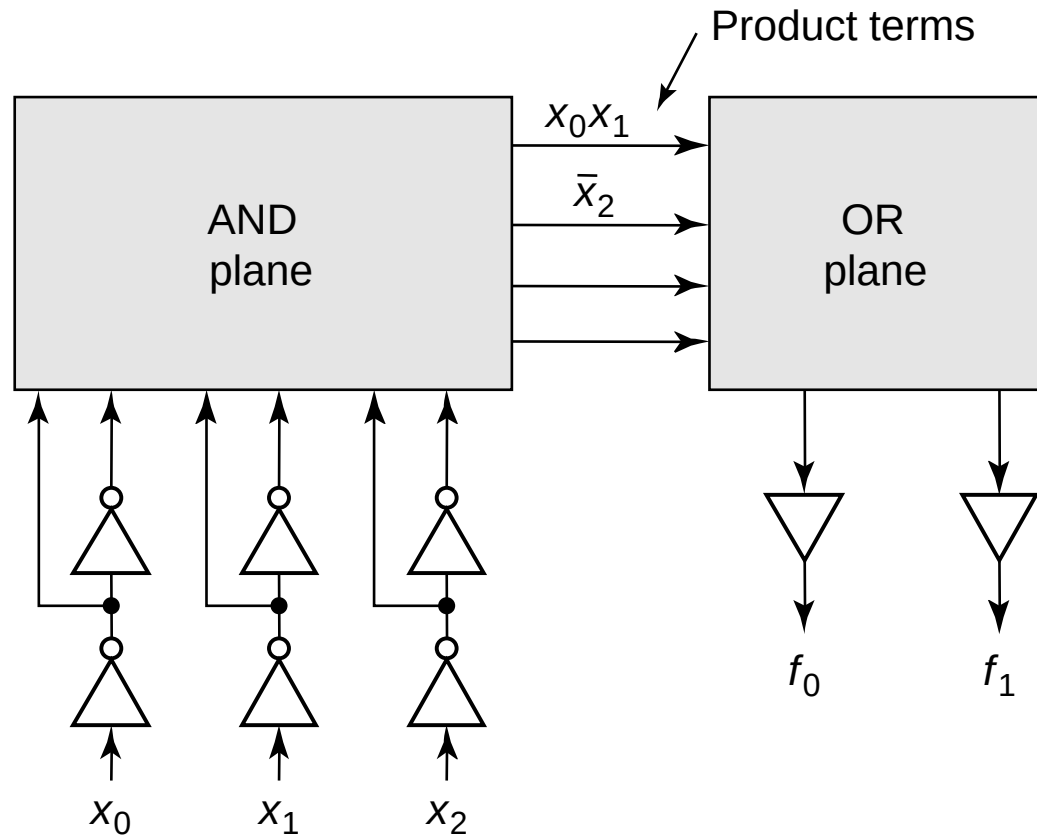
Logic Optimization methods

Depend on target technology



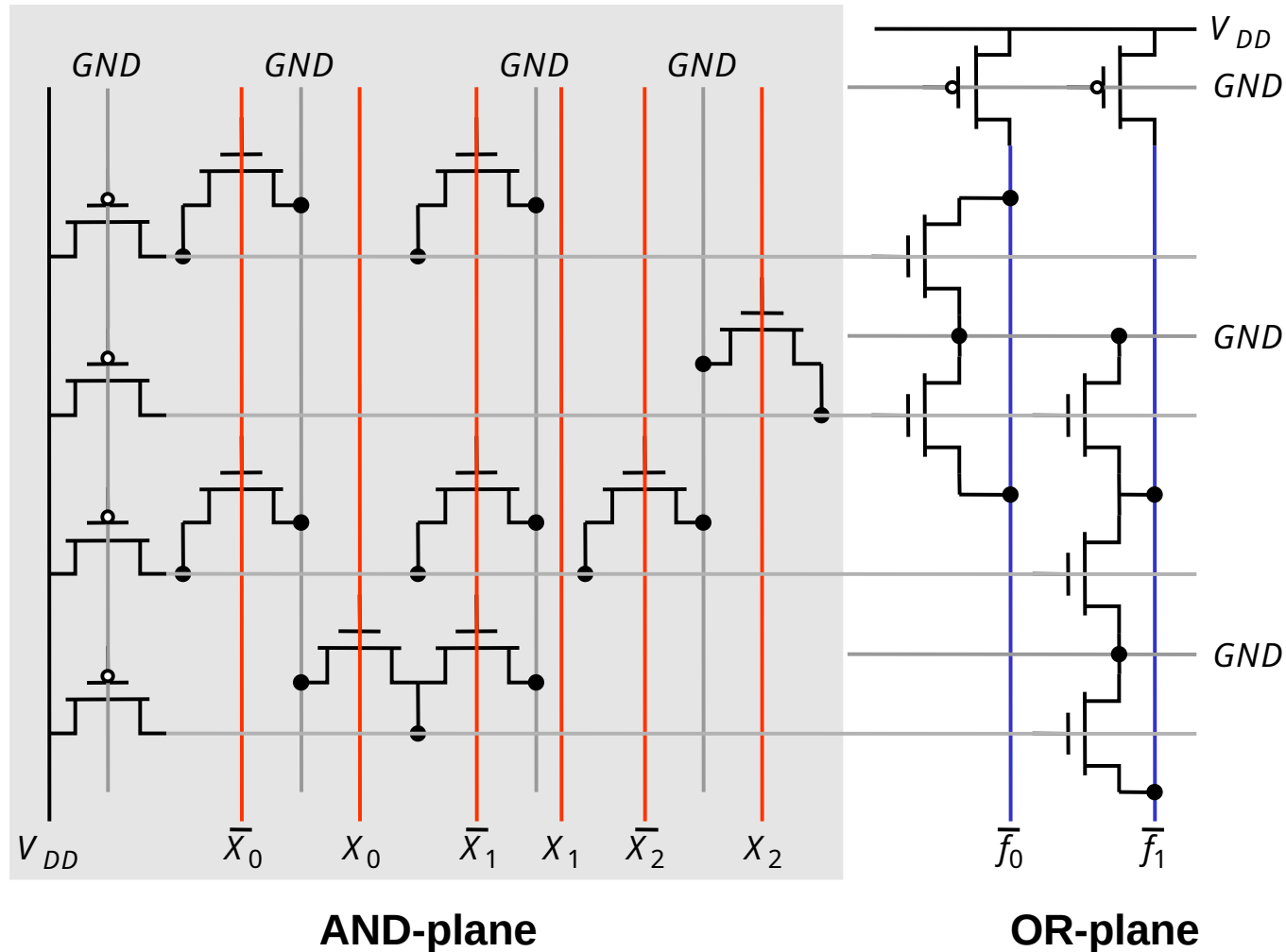
Two-level Logic: PLA

- Logic represented as a two-level AND-OR structure



Programmable Logic Array (PLA)

Pseudo-NMOS PLA



Two-level logic minimization

- **Representation** (which are canonical ?)
 - Truth tables
 - Karnaugh maps
 - Sum of Products (SOP) form
 - Represents number of lines in PLA
 - Binary Decision Diagrams (BDD)
- **Objective**
 - Minimize number of product terms in SOP
 - Challenge: multiple-output functions
- **Optimization techniques**
 - Quine McCluskey (optimal)
 - *Espresso* logic minimizer (heuristic)
 - Ashenhust-Curtis functional decomposition (~optimal)
 - BDD-based (heuristic)

Truth Table

The truth table of a function $f : B^n \rightarrow B$ is a tabulation of its values at each of the 2^n vertices of B^n . (all minterms)

Example: $f = a'b'c'd + a'b'cd + a'bc'd + ab'c'd + ab'cd + abc'd + abcd' + abcd$

(Notation for complement: $a' = \overline{a}$)

The truth table representation is

- **canonical**: if two functions are the same, their representations are the same (isomorphic).
- **intractable for large n**

	<u><i>abcd</i></u>	<u><i>f</i></u>
m_0	0000	0
m_1	0001	1
m_2	0010	0
m_3	0011	1
m_4	0100	0
m_5	0101	1
m_6	0110	0
m_7	0111	1
m_8	1000	0
m_9	1001	1
m_{10}	1010	0
m_{11}	1011	1
m_{12}	1100	0
m_{13}	1101	1
m_{14}	1110	1
m_{15}	1111	1

Karnaugh Maps

- Graphical representation of collection of minterms
 - Two adjacent cells differ in one bit

$$F(w,x,y,z) = \Sigma(0,1,2,4,5,6,8,9,12,13,14) = y' + w'z' + xz'$$

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6
m_{12}	m_{13}	m_{15}	m_{14}
m_8	m_9	m_{11}	m_{10}

		yz		y	
		0 0	0 1	1 1	1 0
w	x				
0 0		1	1		1
0 1		1	1		1
1 1		1	1		1
1 0		1	1		

K-map representation is

- canonical
- impractical for number of variables $n > 5$

Sum of Products (SOP)

Example:

$$abc' + a'bd + b'd' + b'e'f \quad (\text{sum of cubes})$$

Advantages:

- easy to manipulate and minimize
- many algorithms available
- two-level theory applies

Disadvantages:

- Not representative of logic complexity. For example:

$$f = ad + ae + bd + be + cd + ce \quad f' = a'b'c' + d'e'$$

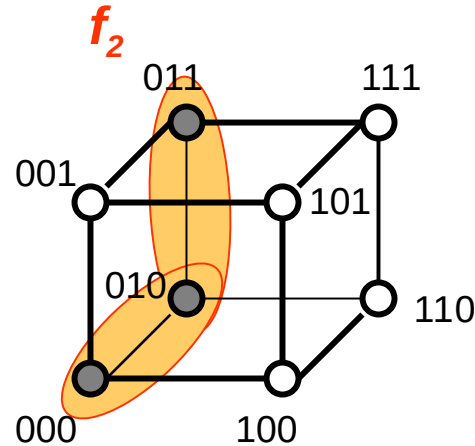
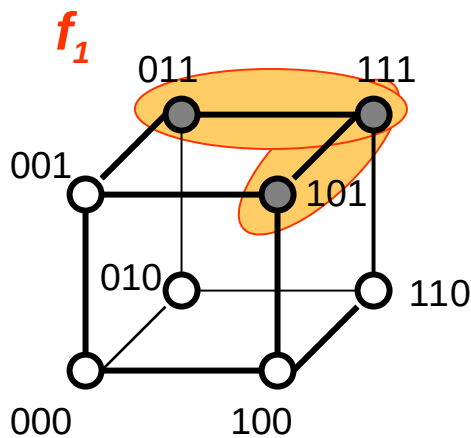
The two differ in their implementation by an inverter.

- Not easy to estimate logic size and performance
- Difficult to estimate progress during logic manipulation

Two-level minimization - basic idea

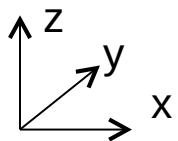
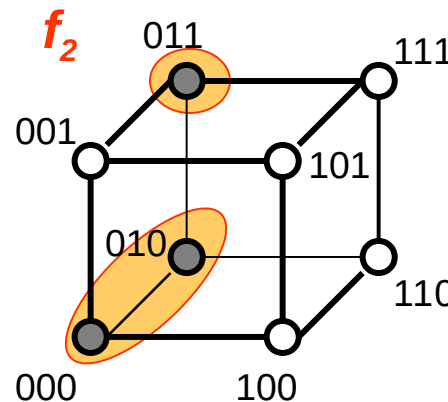
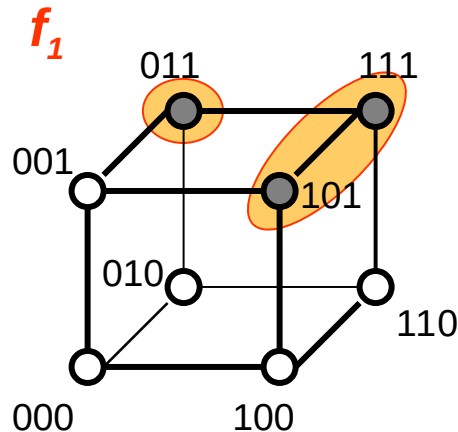
Initial representation:

$x y z$	f_1	f_2
0 - 0	0	1
0 1 -	0	1
- 1 1	1	0
1 - 1	1	0

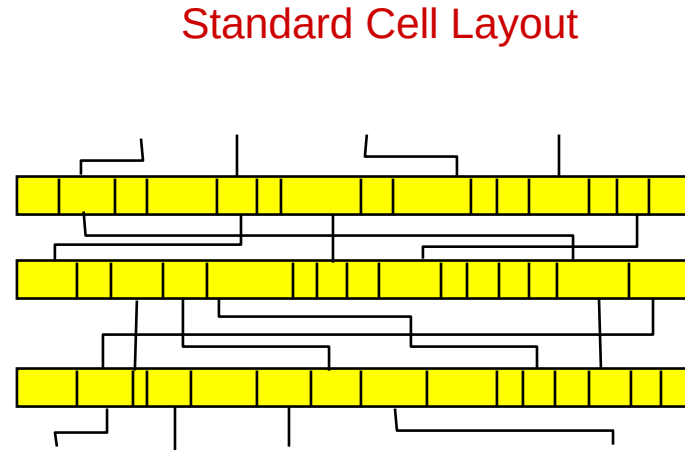
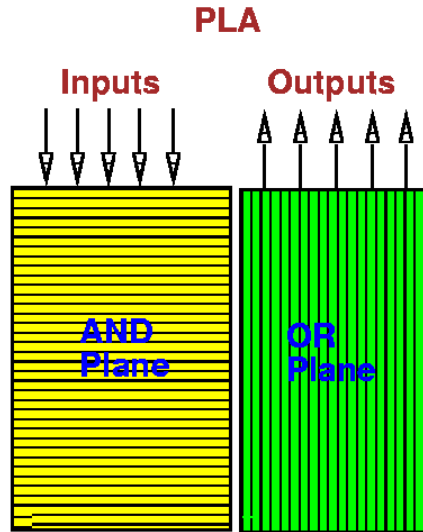


Minimized function:

$x y z$	f_1	f_2
0 - 0	0	1
0 1 1	1	1
1 - 1	1	0



Two-Level (PLA) vs. Multi-Level



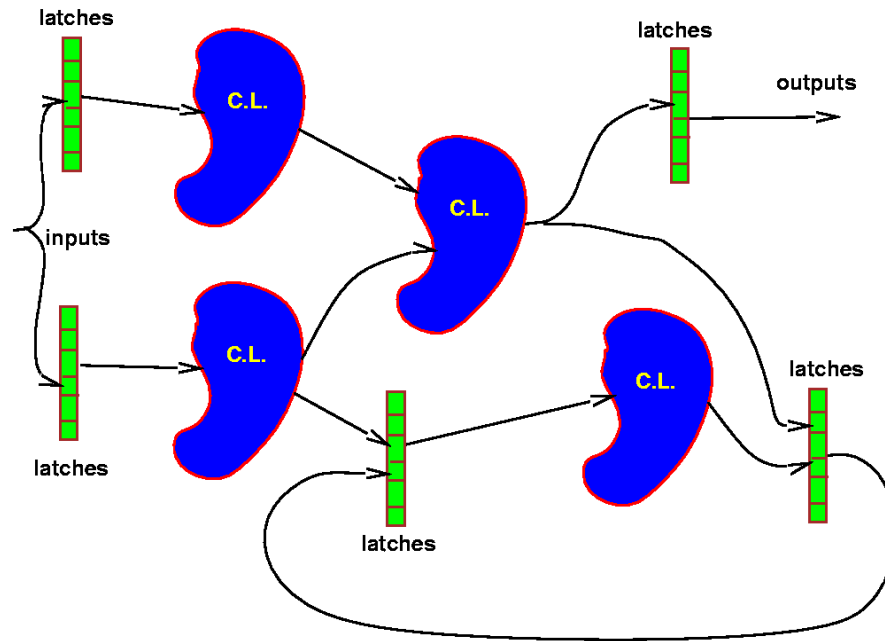
PLA

- control + random logic
- constrained layout, PLA
- goal: minimize # prod. terms

Multi-level Logic

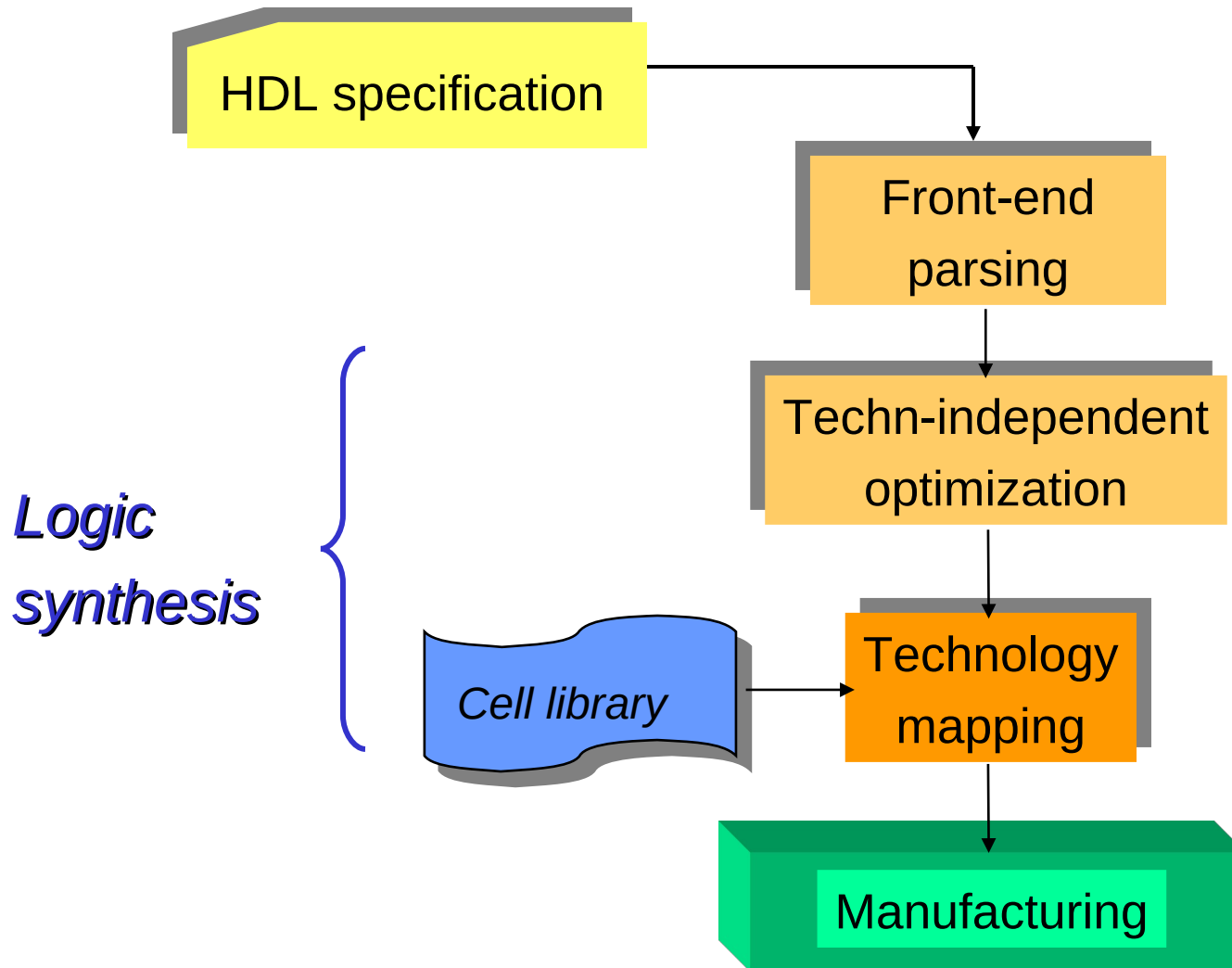
- all logic
- standard cells, FPGAs
- Minimize # gates, transistors (~literals)

General Multi-level Logic Structure

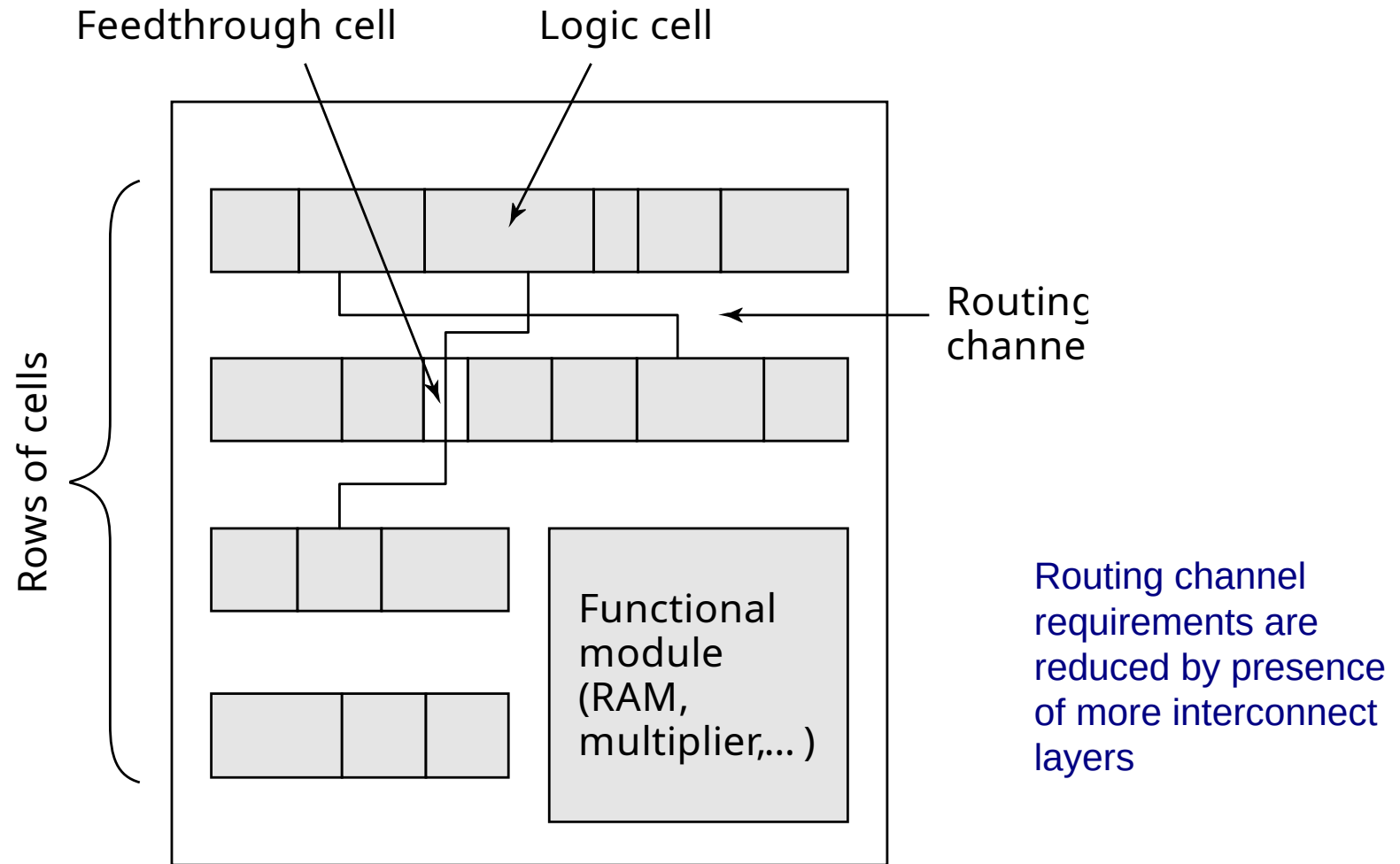


- **Combinational optimization**
 - keep latches/registers at current positions, keep their function
 - optimize combinational logic between register boundaries
- **Sequential optimization**
 - change latch position/function (retiming) + other transformations

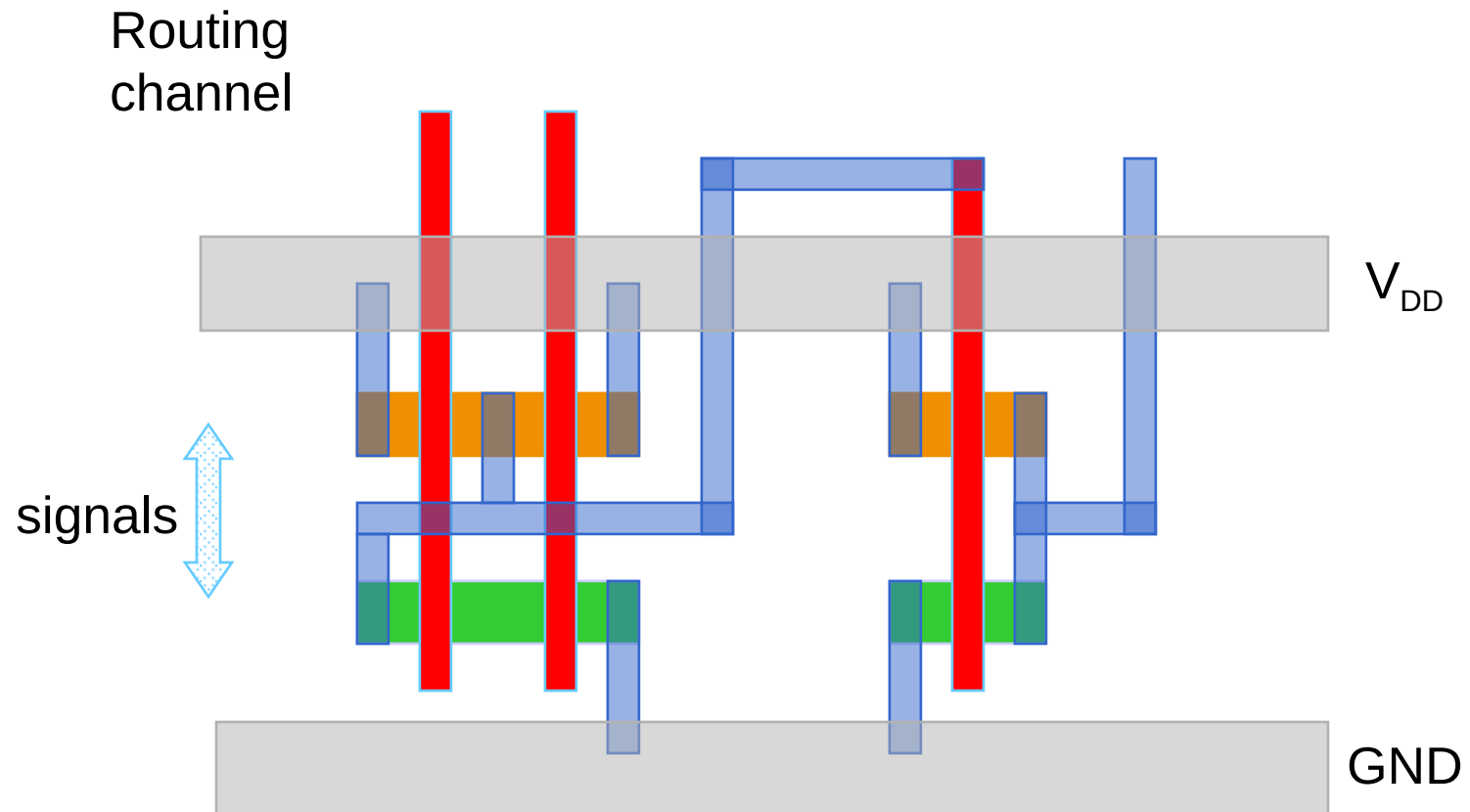
Multi-level logic - Synthesis Flow



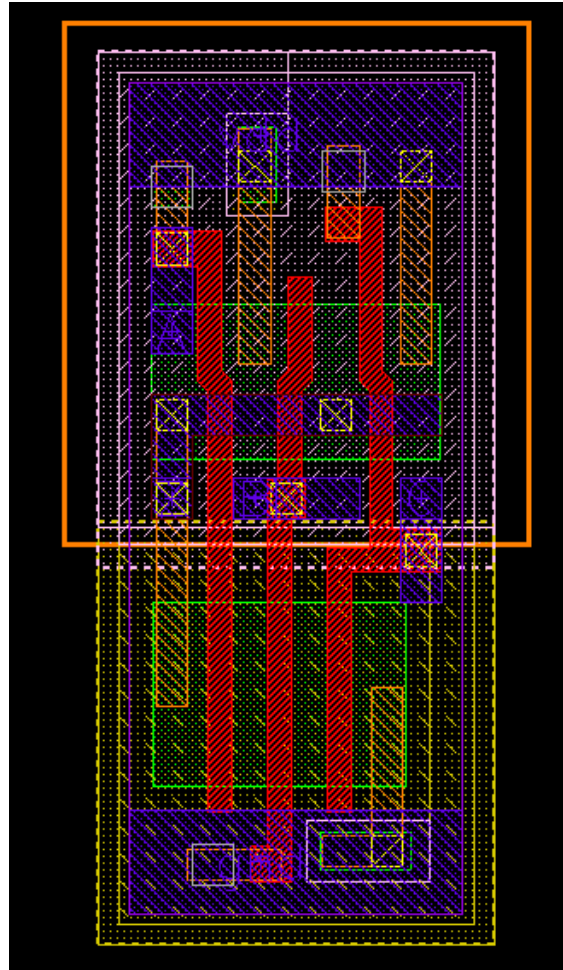
Cell-based Design (standard cells)



Standard Cell Layout Methodology – 1980s



Standard Cell - Example



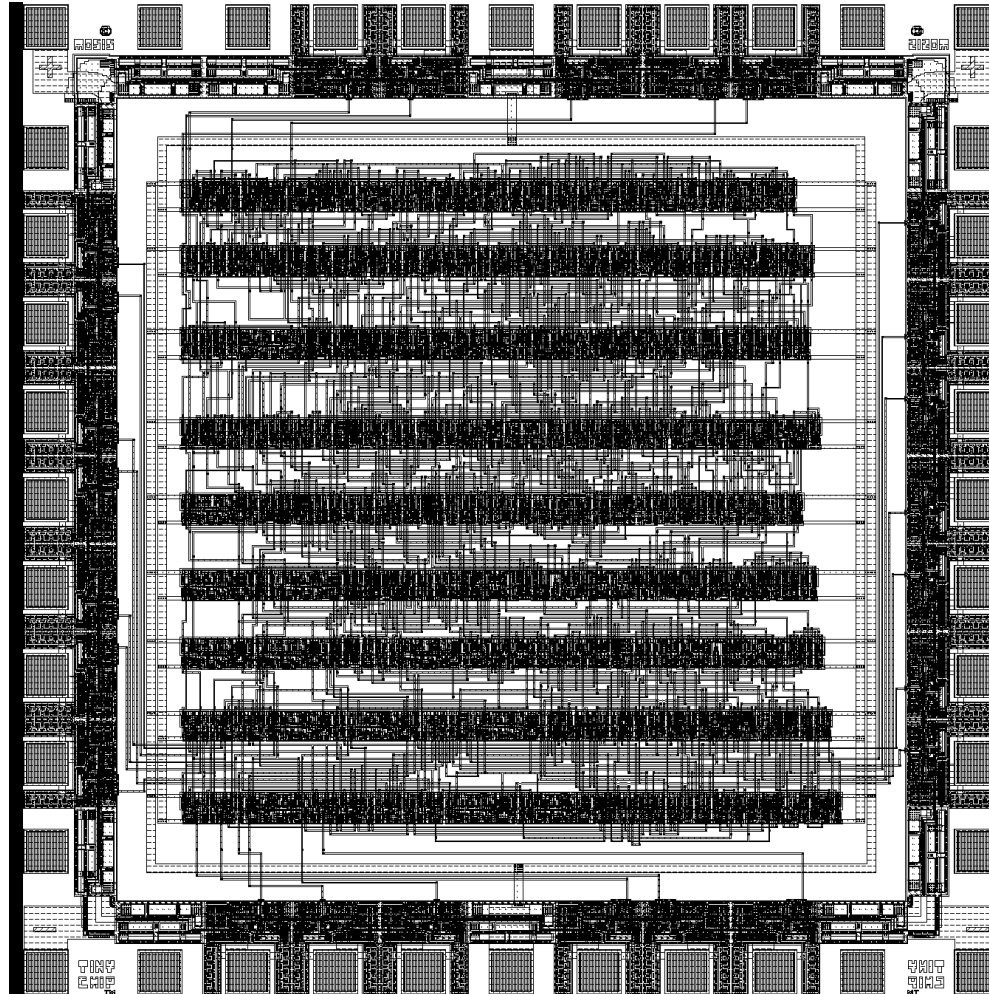
Path	1.2V - 125°C	1.6V - 40°C
$In1-t_{pLH}$	$0.073+7.98C+0.317T$	$0.020+2.73C+0.253T$
$In1-t_{pHL}$	$0.069+8.43C+0.364T$	$0.018+2.14C+0.292T$
$In2-t_{pLH}$	$0.101+7.97C+0.318T$	$0.026+2.38C+0.255T$
$In2-t_{pHL}$	$0.097+8.42C+0.325T$	$0.023+2.14C+0.269T$
$In3-t_{pLH}$	$0.120+8.00C+0.318T$	$0.031+2.37C+0.258T$
$In3-t_{pHL}$	$0.110+8.41C+0.280T$	$0.027+2.15C+0.223T$

3-input **NAND** cell (ST Microelectronics):

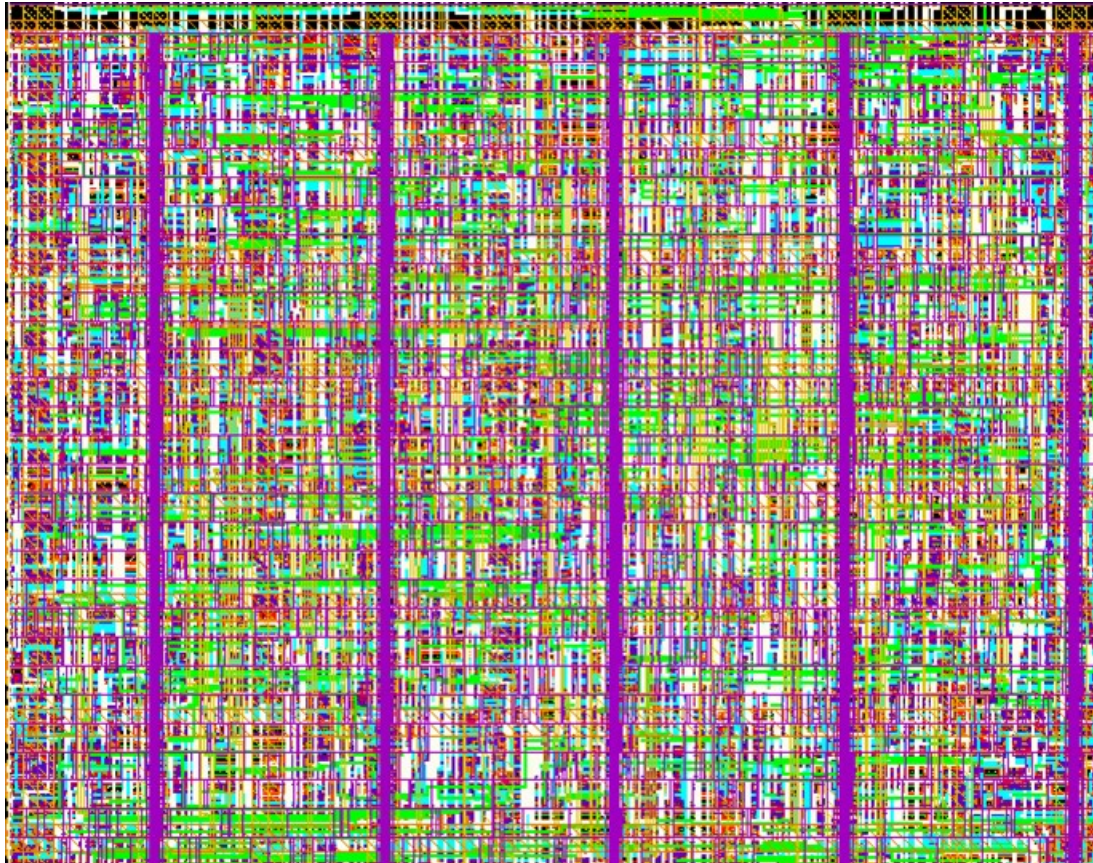
C = Load capacitance

T = input rise/fall time

Standard Cell layout — Example

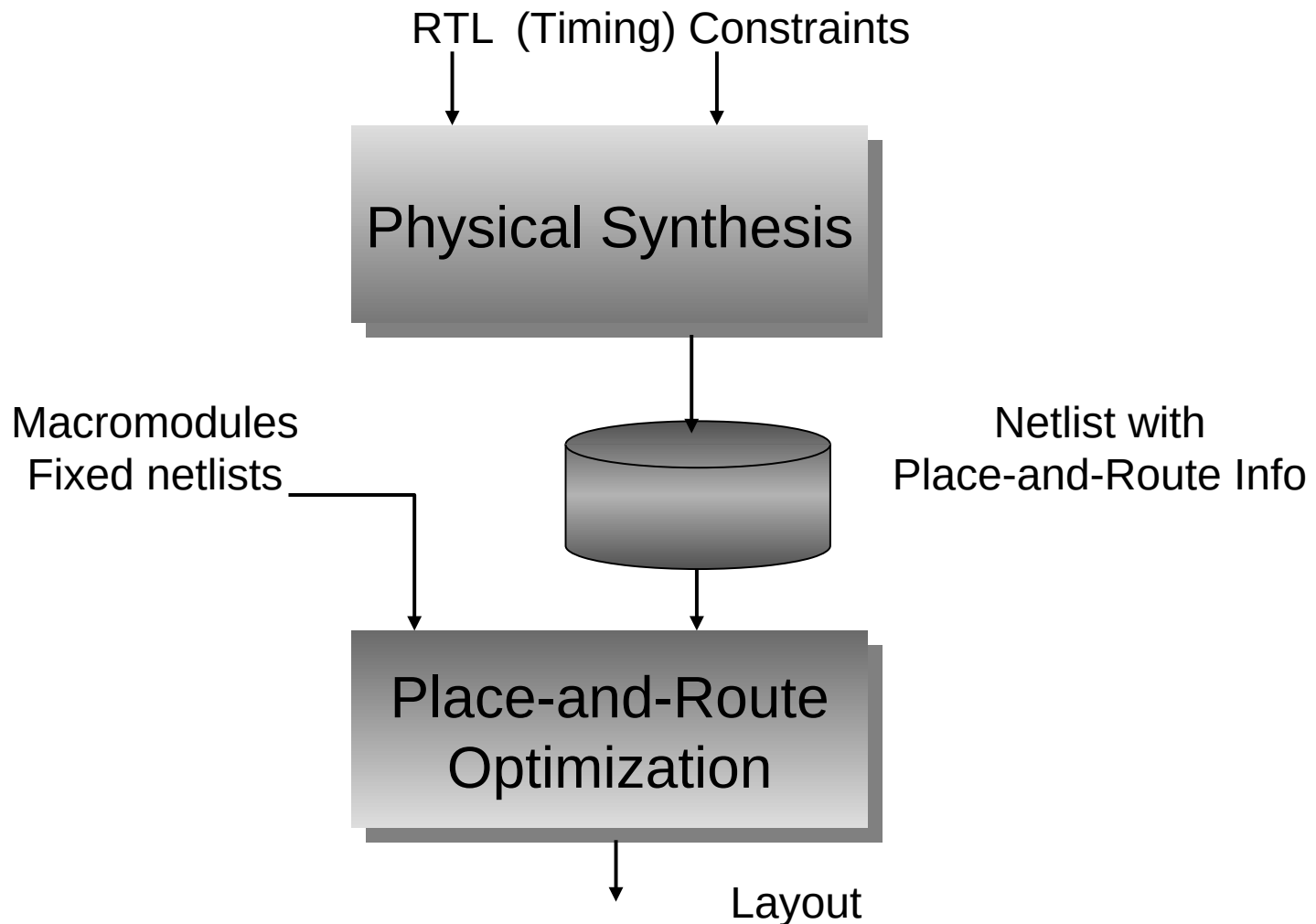


Standard Cell – New Generation

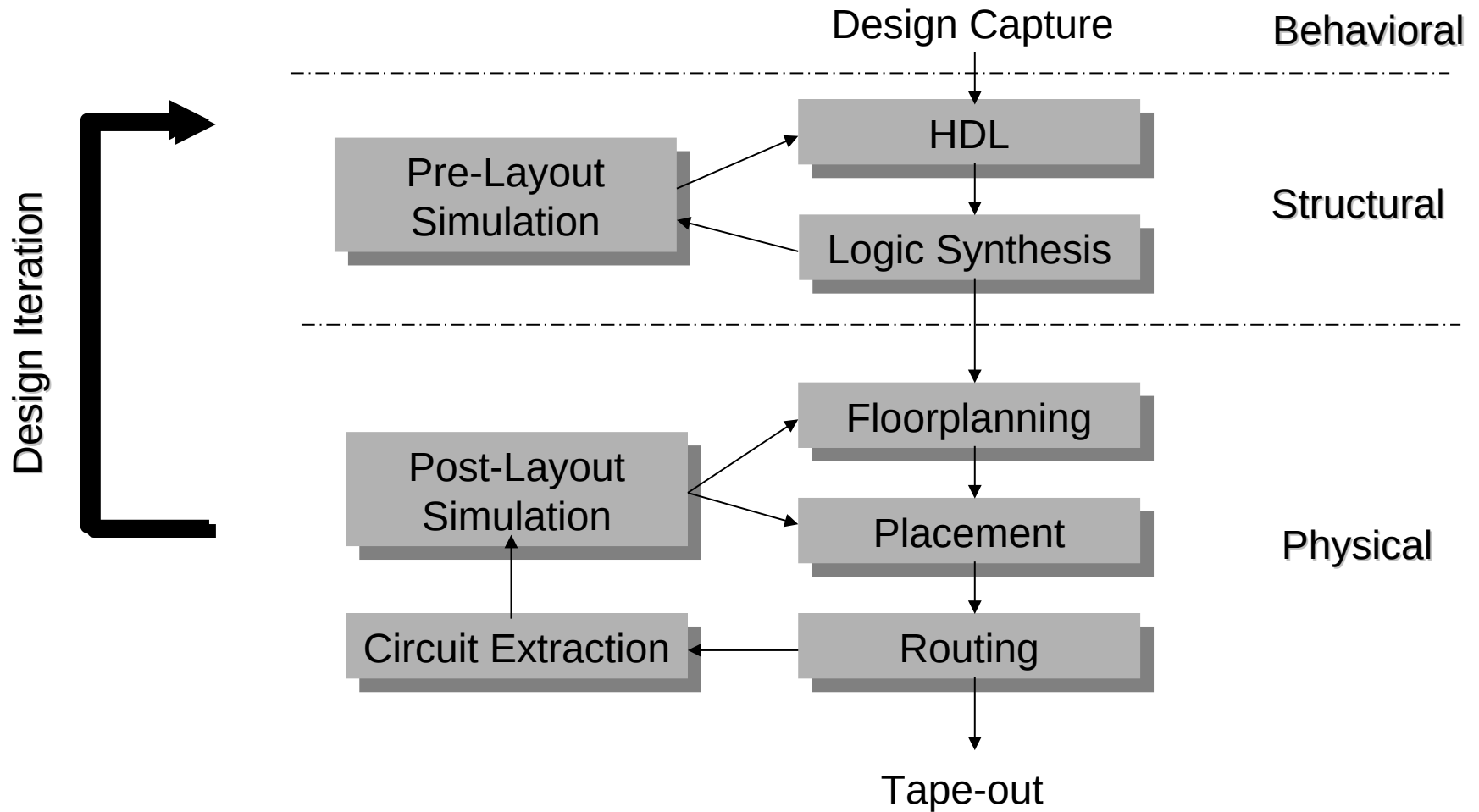


*Cell-structure
hidden under
interconnect layers*

Integrating Synthesis with Physical Design



Semicustom Design Flow

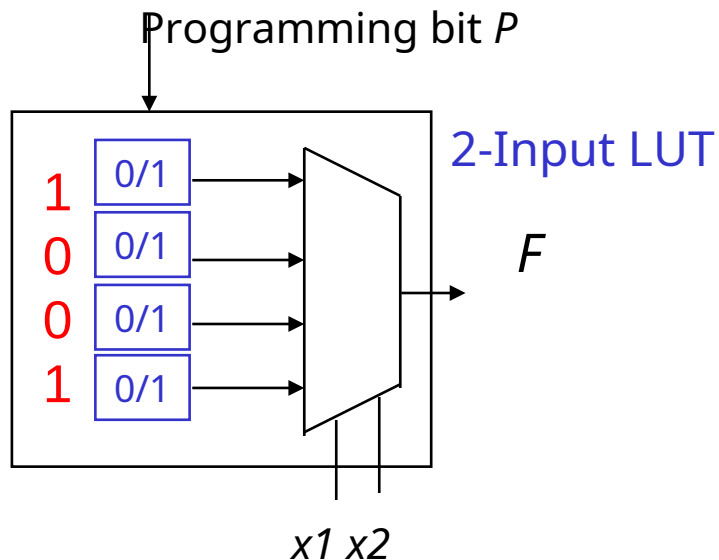


Field Programmable Gate Arrays (FPGA)

- **Field Programmable Gate Array (FPGA)**
 - An array of identical, programmable logic function blocks
 - Manufactured ahead of time (prefabricated)
 - Each block has a fixed number of inputs (k)
 - Each block is able to implement an arbitrary logic function
 - Customer programs FPGA after manufacturing, “in field”
 - provides logic functions and interconnections
 - Re-programmable
 - Easier to debug and cheaper in smaller quantity than ASIC
- **An alternative to ASIC**
 - High production cost amortized over large quantity of chips
 - ASIC (Application Specific Integrated Circuit) – high volume of custom design chip
 - FPGAs – high volume of programmable, flexible chips

Look-up Table based FPGA

- Look-up Table
 - Truth table implemented in hardware
 - Can implement arbitrary function with fixed number of inputs (typically 4-5) by programming the storage bits (customizing the truth table)

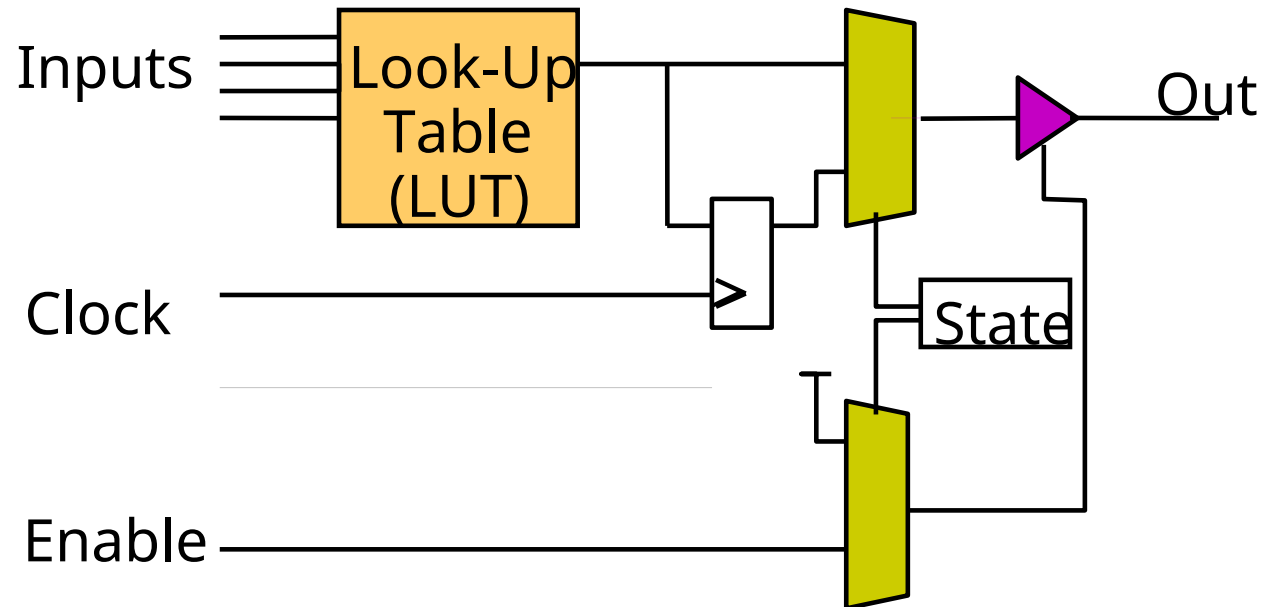


$$F = x_1'x_2' + x_1x_2$$

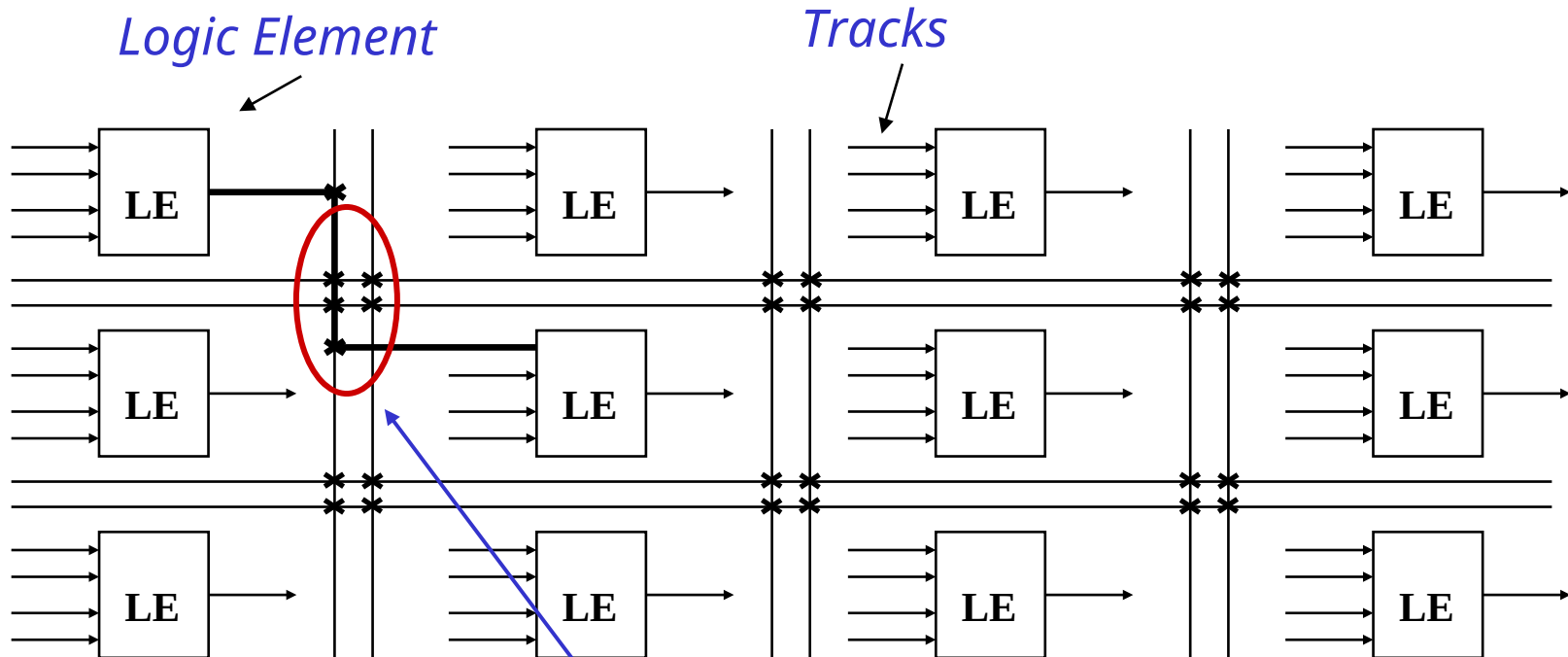
$x1$	$x2$	F
0	0	1
0	1	0
1	0	0
1	1	1

Logic Element

- Logic Element: the basic programmable element of FPGA
 - Contains LUT
- Programming is a domain of specialized technology mapping onto device specific structure



FPGA Architecture



- Each programmable *logic element* outputs one data bit
- Interconnects are also programmable
- A domain of **physical synthesis** (place and route)

Multi-level logic minimization

- **Objective**
 - Minimize number of literals
 - Literals represent inputs to CMOS gates
- **Representation**
 - Factored form
 - Compatible with CMOS
- **Optimization techniques**
 - Algebraic factorization and decomposition (heuristic)
 - Technology independent
 - Requires mapping onto target architecture
 - Standard cells
 - FPGAs (LUT)

Optimization Criteria for Synthesis

- **Objective: minimize some function of:**
 - Area occupied by the logic gates and interconnect (approximated by literals = transistors in technology independent optimization)
 - Critical path delay of the longest path through logic
 - Degree of testability of the circuit
 - Power consumed by the logic gates
 - Placeability, Wireability

Transformation-based Synthesis

- **Synthesis = sequence of transformations** that change network topology and its characteristics
 - All modern synthesis systems are build that way
 - work on uniform network representation
 - use *scripts*, lists of transformations forming a strategy
 - Transformations are mostly *algebraic* !
(very little is based on Boolean factorization)
- **Representation**
 - Cube notation, BDDs, AIGs
- **The underlying algorithms**
 - Algebraic transformations
 - Collapsing, decomposition
 - Factorization, substitution
 - Transformations differ in scope
 - Local (node optimizattion)
 - Global (network restructuring)

Network Representation

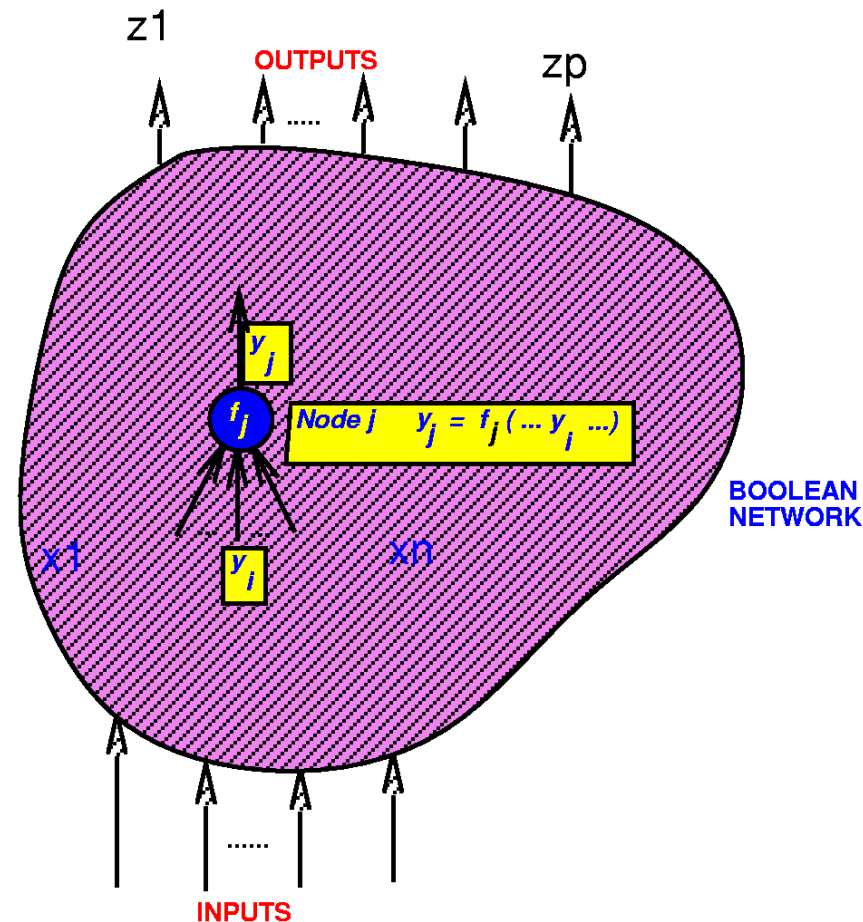
Boolean network:

- directed acyclic graph (DAG)
- **node** logic function representation $f_j(x,y)$
- node variable y_j : $y_j = f_j(x,y)$
- **edge** (i,j) if f_j depends explicitly on y_i

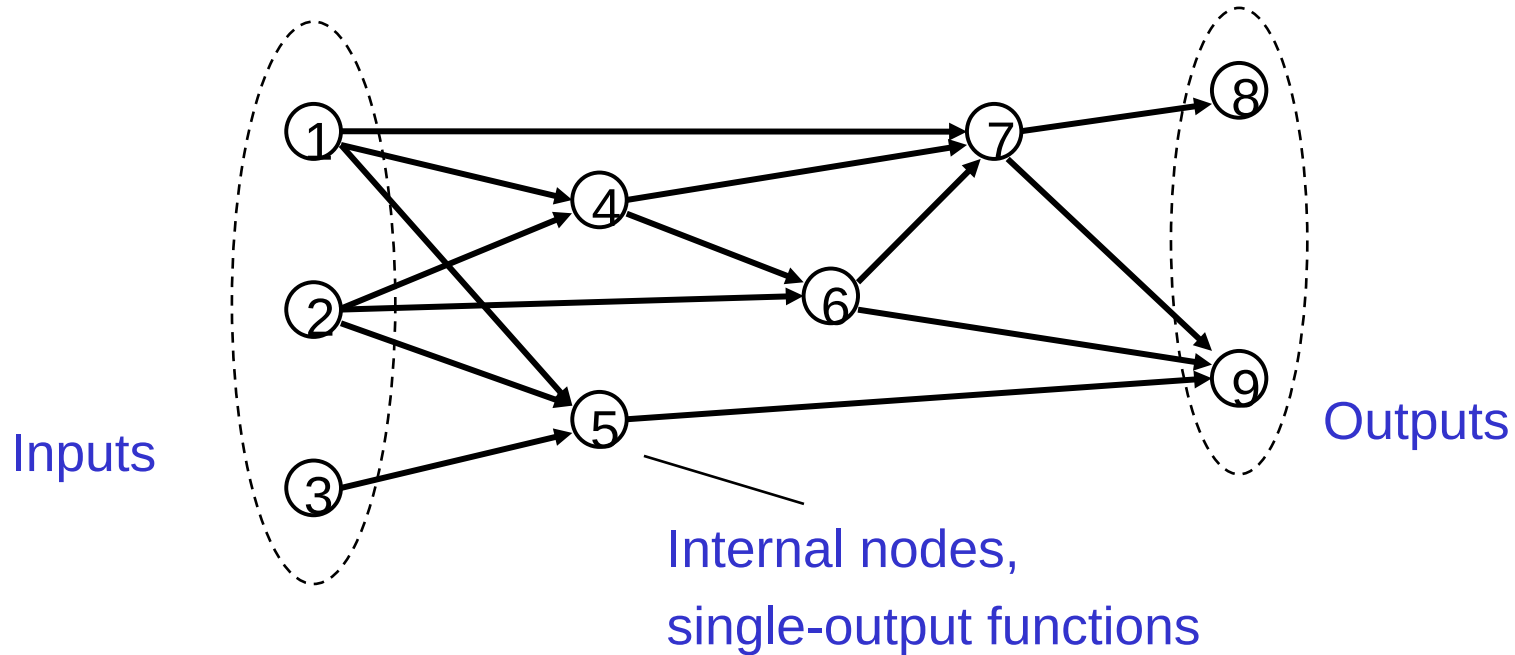
Inputs $x = (x_1, x_2, \dots, x_n)$

Outputs $z = (z_1, z_2, \dots, z_p)$

External don't cares: $d_1(x), \dots, d_p(x)$



Multi-level logic representation: Boolean network



- Goal: minimize some measure of network complexity
 - number of 2-input gates
 - number of literals (variables)
- Eventually, the nodes must be mapped to standard cells (technology mapping)

Sum of Products (SOP)

Used to represent local Boolean functions (nodes)

$$abc' + a'bd + b'd' + b'e'f \quad (\text{sum of cubes})$$

Advantages:

- easy to manipulate and minimize
- many algorithms available (e.g. AND, OR, TAUTOLOGY)

Disadvantages:

- Not representative of logic complexity. For example:

$$f = ad + ae + bd + be + cd + ce \quad f' = a'b'c' + d'e'$$

These differ in their implementation by an **inverter**.

- Not easy to **estimate** logic size and performance

Factored Forms

Example:

$$(ad+b'c)(c+d'(e+ac'))+(d+e)fg$$

Advantages

- good representative of logic complexity
 $f=ad+ae+bd+be+cd+ce \Rightarrow f=(a+b+c)(d+e)$
- in many designs (e.g. complex gate CMOS) the implementation of a function corresponds directly to its factored form
- good estimator of logic implementation complexity
- doesn't blow up easily

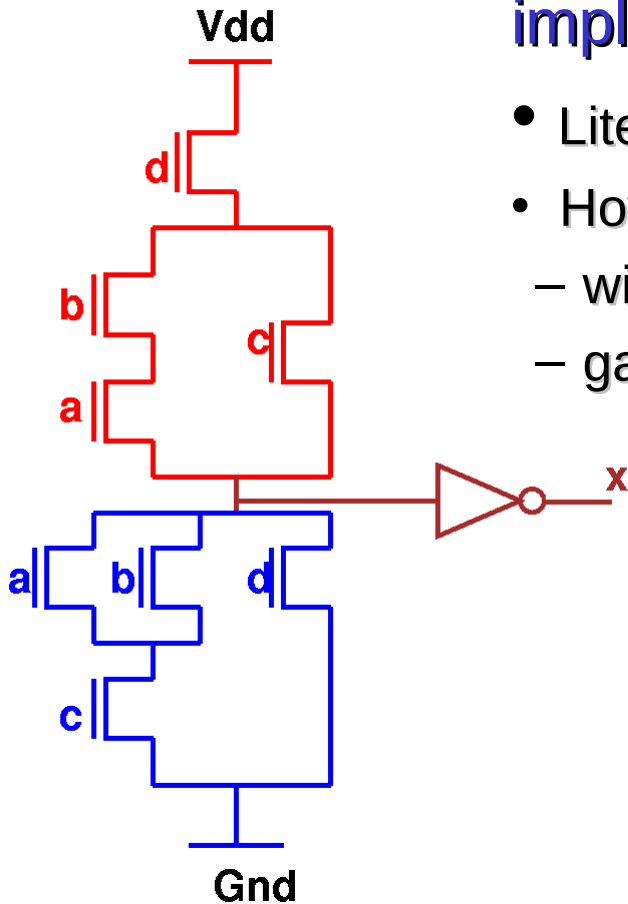
Disadvantages

- not as many algorithms available for manipulation
- often just converted into SOP before manipulation

Factored Forms

Good approximation for multi-level logic implemented in CMOS

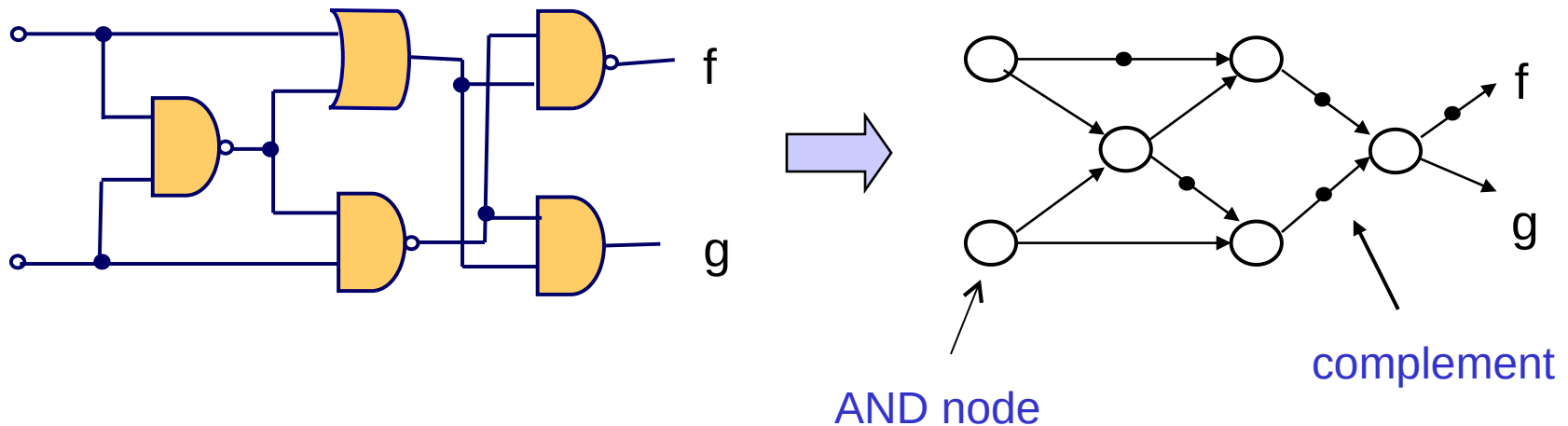
- Literal count $\approx$ transistor count $\approx$ area
- However, area also depends on
 - wiring
 - gate size etc.



$$X = (a+b)c + d$$

AND-INVERTER Graphs (AIG)

- New representation, for state of the art synthesis (ABC system)
- Base data structure uses two-input *AND* function for vertices and *Inverter* attributes at the edges (individual bit)
 - use De'Morgan's law to convert OR operation etc.
- Hash table to identify and reuse structurally isomorphic circuits



Logic Optimization (techn-independent)

Goal: given initial network, find *best* factored form representation.

Example: $f_1 = abcd + abce + ab'cd' + ab'c'd' + a'c + cdf + abc'd'e' + ab'c'df'$

$$f_2 = bdg + b'dfg + b'd'g + bd'eg$$

- SOP minimization (2-level)

$$f_1 = bcd + bce + b'd' + a'c + cdf + abc'd'e' + ab'c'df'$$

$$f_2 = bdg + dfg + b'd'g + d'eg$$

- Factoring

$$f_1 = c(b(d+e) + b'(d'+f) + a') + ac'(bd'e' + b'df')$$

$$f_2 = g(d(b+f) + d'(b'+e))$$

- Decomposition

$$f_1 = c(x+a') + ac'x' \quad , \quad f_2 = gx$$

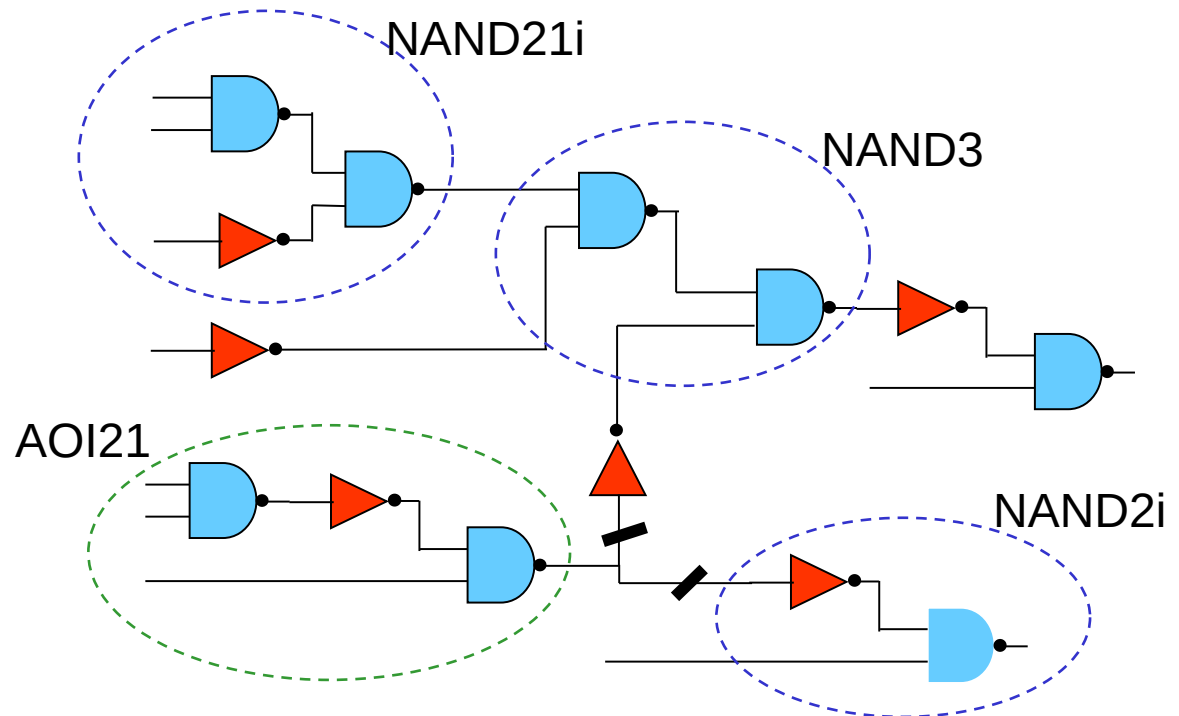
$$x = d(b+f) + d'(b'+e)$$

Logic optimization tasks:

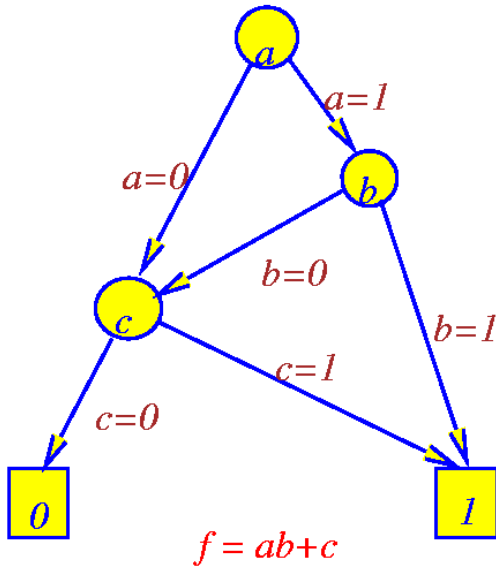
- find good **common** subfunctions
- effect the **division**

Technology dependent Optimization

- Logic represented as a network of logic gates
 - Logic decomposition (multi-level network)
 - Technology mapping onto standard cells (library)

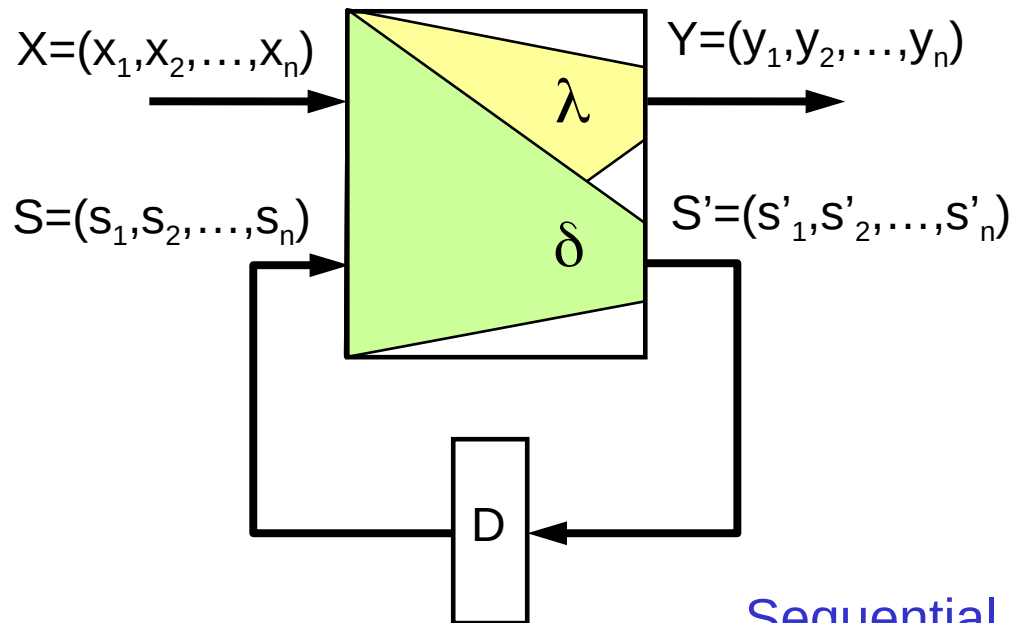


Binary Decision Diagrams (BDDs)



- Like factored form, represents both function and **complement**
- Like network of muxes, but restricted since controlled by **primary input** variables
 - **not really a good estimator** for implementation complexity
- Given an ordering, reduced BDD is **canonical**, hence a good replacement for truth tables
- For a good **ordering**, BDDs remain reasonably small for complicated functions (e.g. not multipliers)
- **Manipulations** are well defined and efficient
- **True** support (dependency) is displayed

Basic Model of Sequential Circuit: FSM



$M(X, Y, S, S_0, \delta, \lambda)$:

X : Inputs

Y : Outputs

S : Current State

S_0 : Initial State(s)

δ : $X \times S \rightarrow S$ (next state function)

λ : $X \times S \rightarrow Y$ (output function)

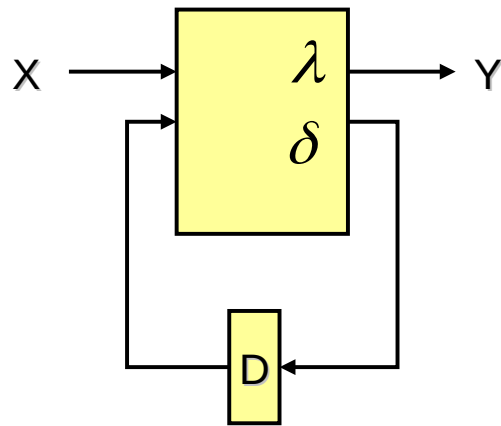
Sequential synthesis:

find (multi-level) implementation of $\delta(X)$ and $\lambda(X)$ that minimize its cost (area, delay, power)

Delay elements:

- **Clocked**: synchronous
 - single-phase clock, multiple-phase clocks
- **Unclocked**: asynchronous

Sequential Logic Synthesis (FSM view)



Given: Finite-State Machine $F(X, Y, Z, \lambda, \delta)$ where:

X : Input alphabet

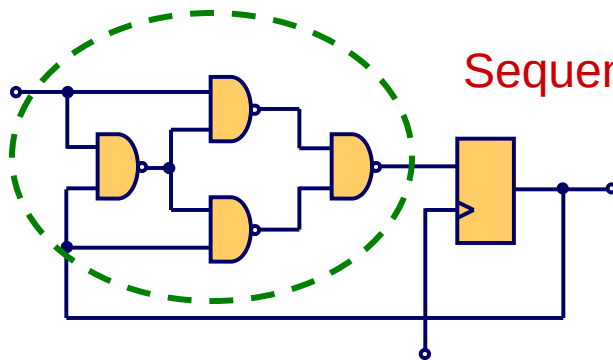
Y : Output alphabet

Z : Set of internal states

$\lambda : X \times Z \rightarrow Z$ (next state function, *Boolean*)

$\delta : X \times Z \rightarrow Y$ (output function, *Boolean*)

Combinational logic

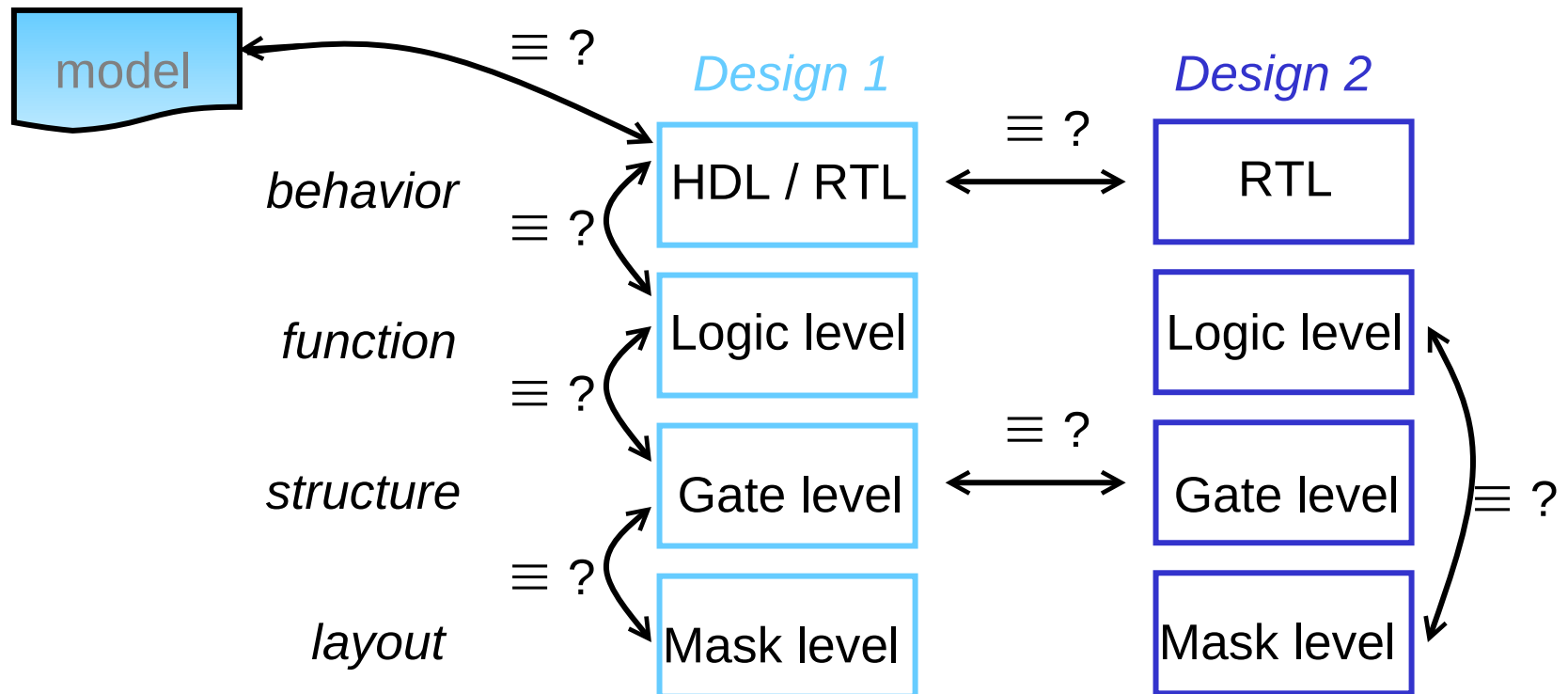


Sequential elements

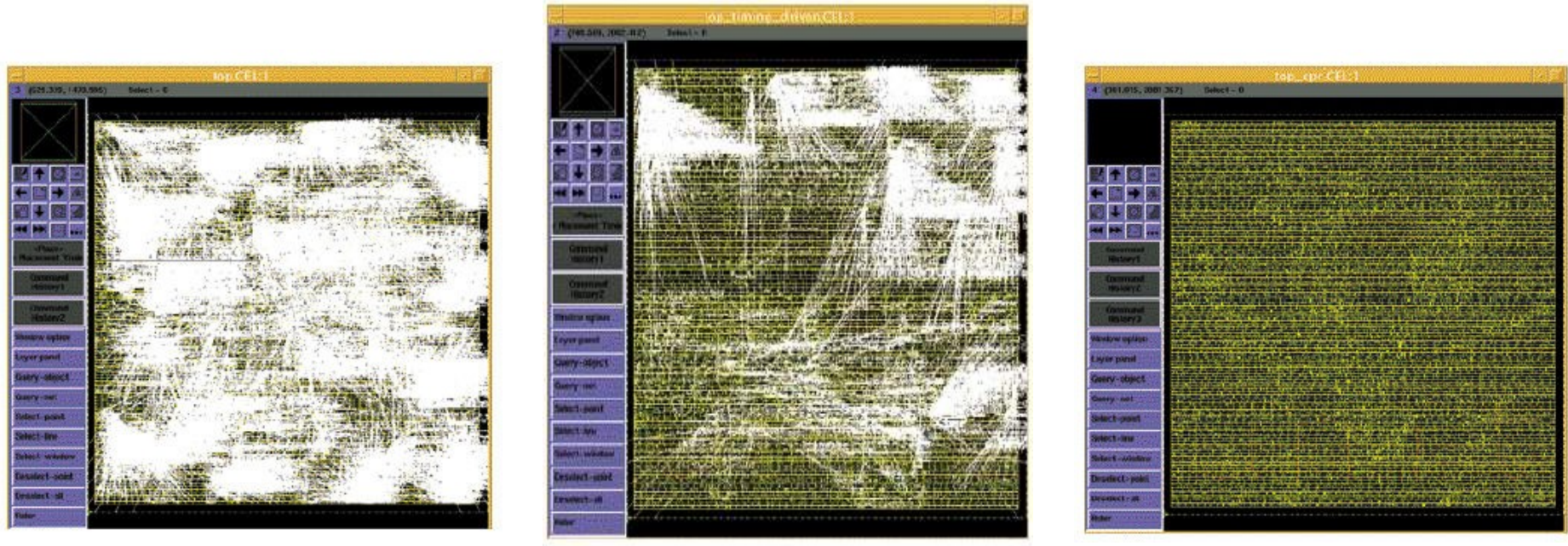
Circuit composed of interconnected set of Boolean gates, flip-flops, latches, registers, etc.

Verification

- **Design verification = ensuring correctness of the design**
 - against its implementation (at different levels)
 - against alternative design (at the same level)



The “Design Closure” Problem



Iterative Removal of Timing Violations (white lines)